



**ESCUELA TÉCNICA SUPERIOR
DE
INGENIERIA INFORMÁTICA**

Trabajo Fin de Máster

Máster en Lógica, Computación, e Inteligencia Artificial

Sing2Text: Traducción basada en KeyPoints

Realizado por

Nicolás Rodríguez Ruiz

Tutorizado por

Miguel Ángel Martínez Del Amor

Departamento de

Ciencias de la Computación e Inteligencia Artificial

Sevilla, 29 de junio de 2026

Índice general

1	Introducción	11
1.1	Motivación y Planteamiento	11
1.2	Objetivo	12
1.3	Estructura del documento	13
2	Estado del Arte	15
2.1	Evolución de la traducción de lengua de signos	15
2.2	Modelos basados en glosas (dos etapas)	16
2.2.1	Ventajas del enfoque	17
2.2.2	Desventajas: propagación de errores	17
2.3	Modelos libre de glosas (una etapa)	18
2.3.1	El límite de la traducción directa	18
2.4	El renacimiento de las glosas mediante Modelos de Lenguaje (LLMs) . . .	19
2.4.1	Traducción híbrida: el enfoque Spotter+GPT	19
2.4.2	Limitaciones de los LLMs masivos en SLT	20
2.5	Modelos basados en esqueletos (<i>keypoints</i>)	20
2.5.1	Atención multiescala: El modelo MSKA	21
2.6	Conjuntos de datos	21
2.6.1	Selección del conjunto de datos	23
3	Fundamentos	25
3.1	Medidas de precisión en traducción	25
3.1.1	WER	25
3.1.2	BLEU	25
3.1.3	ROUGE-L	26
3.1.4	BLEURT	26

3.2	Transformers	26
3.2.1	Mecanismo de atención	27
3.2.2	Codificación posicional	27
3.3	Clasificación Temporal Conexionista	28
3.4	<i>Decoupled Spatial-Temporal Attention</i> (DSTA)	29
3.4.1	Módulos de atención espacial y temporal	30
3.4.2	Codificación Posicional	32
3.4.3	<i>Spatial Global Regularization</i> (SGR)	33
3.5	LoRA	35
4	Metodología	37
4.1	Extracción de <i>Keypoints</i>	37
4.1.1	El ecosistema MMPose y la adopción de AlphaPose	38
4.1.2	Configuración y arquitectura de AlphaPose	38
4.2	Normalización de los Datos	41
4.3	<i>Multi-Stream Keypoint Attention</i>	41
4.3.1	Data Augmentation	41
4.3.2	Arquitectura del Modelo de Reconocimiento	44
4.3.3	Arquitectura del módulo de traducción	48
4.3.4	Tokenizador	51
5	Experimentación y análisis de resultados	53
5.1	Configuración experimental	53
5.1.1	Entorno de entrenamiento	53
5.1.2	Hiperparámetros generales	54
5.1.3	Configuración de LoRA	54
5.1.4	Configuración de generación	55
5.2	Análisis exploratorio del dataset	55
5.2.1	Estadísticas del conjunto de entrenamiento	56
5.2.2	Vocabulario de glosas	56
5.3	Reconocimiento de Lengua de Signos (SLR)	56
5.3.1	Modelo base	57
5.3.2	Estudio de aumentación de datos	58
5.4	Traducción mediante mBART	61

5.4.1	Resultados y análisis	61
5.5	Traducción <i>zero-shot</i> : Spotter + Qwen	62
5.5.1	Configuración	62
5.5.2	Resultados y análisis	63
5.6	Ablation study: Qwen2.5-1.5B	65
5.6.1	Modalidad de entrada: <i>features</i> visuales vs. glosas predichas	65
5.6.2	Impacto del <i>prompt</i>	66
5.6.3	<i>Finetuning</i> completo vs. LoRA	67
5.6.4	Mejor configuración de Qwen2.5-1.5B	67
5.7	Escalado a Qwen2.5-7B	68
5.8	Comparativa global y discusión	68
5.8.1	Resumen de resultados	68
5.8.2	Discusión	69
5.8.3	Coste computacional	69
6	Conclusión y trabajo futuro	71

Índice de figuras

2.1	Esquema de la arquitectura basada en gloass	16
2.2	Esquema de la arquitectura basada en modelos de una etapa	18
2.3	Estructura del modelo Spotter+GPT propuesto por Sincan et al. [20] . . .	20
3.1	Diagrama de la atención multi-cabeza	27
3.2	Las tres estrategias propuestas en [22]. Grafico sacado de ese papel.	30
3.3	Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un esquema de lo que ocurre.	32
3.4	Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un esquema de lo que ocurre.	33
3.5	Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un esquema de lo que ocurre.	33
3.6	Esquema del módulo de atención espacial . Obtenido de [22]	34
3.7	Esquema de la red. Obtenido de [22]	35
4.1	Distribución de <i>keypoints</i> anatómicos en el formato Halpe-136 [7].	40
4.2	Esquema de la estructura del módulo de reconocimiento del modelo. Imagen de [19]	44
4.3	Esquema de la división de las extremidades. Imagen de [19]	45
4.4	Esquema del módulo de atención de <i>keypoints</i> del cuerpo. Se toma $h = 6$. Imagen de [19]	45
4.5	Esquema las redes de cabecera.	47
5.1	Evolución de la pérdida (izquierda) y del WER en validación (derecha) a lo largo del entrenamiento para las distintas configuraciones de aumentación. Las líneas transparentes en la gráfica de pérdida corresponden a las curvas de entrenamiento. Las estrellas marcan el <i>checkpoint</i> con el mejor modelo.	60

TODO: EConclusión y trabajo futuro

Capítulo 1

Introducción

1.1. Motivación y Planteamiento

Según los datos de la Organización Mundial de la Salud, más de 1,500 millones de personas en el mundo viven con algún grado de pérdida auditiva, de las cuales aproximadamente 430 millones requieren atención especializada. Para una parte significativa de esta población, la lengua de signos es su principal medio de comunicación natural. Sin embargo, el acceso a servicios esenciales como lo son la sanidad, la educación o la administración pública sigue dependiendo casi exclusivamente de intérpretes humanos, un recurso escaso, costoso y con disponibilidad geográfica muy desigual. Esta dependencia limita la autonomía de las personas sordas y perpetúa una brecha de inclusión social difícil de cerrar sin el apoyo de soluciones tecnológicas escalables.

En los últimos años ha habido enormes avances en las técnicas de *Procesamiento del Lenguaje Natural*, *Visión por Computador* y *Aprendizaje Profundo*, así como en la eficiencia y capacidad computacional de los equipos, sin embargo las lenguas de signos han recibido una atención comparativamente menor, y los avances no han alcanzado el nivel de madurez ni la cobertura lingüística que sí presentan los sistemas para lenguas orales. Los sistemas de traducción automática, los asistentes conversacionales y las interfaces de voz han transformado la forma en que las personas oyentes interactúan con la tecnología, pero siguen siendo inaccesibles para quienes se comunican mediante el canal viso-gestual.

Para abordar esta brecha, la investigación en traducción automática de lengua de signos se articula principalmente en torno a dos direcciones complementarias: por un lado, la traducción de texto o voz a lengua de signos, **Text2Sign**, orientada a facilitar la comprensión por parte de personas sordas; por otro, la traducción de lengua de signos a texto, **Sign2Text**, cuyo objetivo es permitir que las personas oyentes o los sistemas automáticos comprendan los mensajes signados. El presente trabajo se centra en esta segunda dirección, **Sign2Text**, por su impacto directo en la autonomía comunicativa de las personas sordas y por los retos técnicos abiertos que aún presenta.

La tarea **Sign2Text**, también conocida como *Sign Language Recognition and Translation*, persigue la generación automática de texto a partir de una secuencia de signos capturada en vídeo. Un sistema de este tipo permitiría subtitular en tiempo real una conversación signada, dotar a los asistentes virtuales de la capacidad de entender a usuarios sordos, o facilitar el acceso a servicios públicos sin necesidad de un intérprete presente. En definitiva, actuaría como un puente de comunicación directo entre la comunidad sorda y los sistemas digitales diseñados, hasta ahora, pensando exclusivamente en usuarios oyentes.

Desde el punto de vista técnico, Sign2Text es un problema especialmente complejo. A diferencia del reconocimiento de voz, donde la señal de entrada es unidimensional y continua, la lengua de signos es inherentemente multimodal: la información se distribuye simultáneamente entre la configuración y el movimiento de las manos, la expresión facial, la postura corporal y el uso del espacio. Esta riqueza expresiva, que dota a las lenguas de signos de toda su potencia comunicativa, supone al mismo tiempo uno de los mayores desafíos para su modelado automático. A esto se le suma la gran variedad de lenguas de signos que existen, cada una con sus gestos y expresiones visuales únicas, además en la mayoría de los casos, la lengua de signos de una determinada región carece de relación gramatical con la lengua natural de dicha región. Esto dificulta significativamente la comprensión y traducción directa entre un lenguaje natural y de signos.

Además de esta complejidad multimodal, la naturaleza visual de la tarea presenta consideraciones relevantes en materia de privacidad. Procesar vídeos de personas requiere garantías sobre el tratamiento de datos personales, especialmente en entornos sensibles como la sanidad o la educación. Por ello, un enfoque que minimice la exposición de datos desde el origen resulta deseable. A raíz de esto nace la traducción de signos basados en *keypoints*, esto es puntos predeterminados en el cuerpo humano que resumen la información de la postura, expresión e información de manos, capturando la información gestual relevante sin preservar la identidad visual de la persona, permitiendo así un procesamiento más ligero, portable y respetuoso con la privacidad. De forma paralela, el uso de este enfoque, reduce drásticamente la dimensionalidad de los datos, optimizando los costes computacionales de entrenamiento e inferencia y facilitando su despliegue en equipos menos potentes.

1.2. Objetivo

El objetivo principal de este trabajo es explorar la viabilidad de sustituir el decodificador de traducción clásico del sistema MSKA-SLT, basado en mBART, por un **Modelo de Lenguaje Pequeño** (*Small Language Model*, SLM) de arquitectura *decoder-only*, con el fin de dotar al sistema de mayor capacidad de razonamiento contextual manteniendo la ejecución local y la privacidad del usuario.

Para alcanzar este objetivo principal, se plantean los siguientes **objetivos específicos**:

1. **Reimplementar y validar el sistema MSKA-SLT como línea base.** Repro-

ducir la arquitectura original de reconocimiento y traducción de lengua de signos basada en MSKA sobre el corpus PHOENIX-2014T, verificando que los resultados obtenidos son coherentes con los publicados en la literatura para establecer un punto de referencia riguroso.

2. **Diseñar e integrar el módulo de proyección visual (*VLMapper*).** Desarrollar el componente encargado de alinear el espacio de representaciones visuales producido por el reconocedor MSKA-SLR con el espacio de *embeddings* del modelo de lenguaje, permitiendo la inyección de información visual como prefijo de contexto en un modelo *decoder-only*.
3. **Evaluar la capacidad de traducción en modo *zero-shot*.** Analizar el comportamiento del SLM sin ningún entrenamiento específico para la tarea, usando únicamente un prompt de instrucción en alemán sobre las glosas predichas por el reconocedor, siguiendo el paradigma del trabajo Spotter+GPT, para caracterizar el límite inferior del sistema antes de cualquier adaptación.
4. **Adaptar el SLM mediante *finetuning* eficiente con LoRA.** Entrenar el sistema completo sobre PHOENIX-2014T aplicando adaptadores de bajo rango (*Low-Rank Adaptation*) en distintas configuraciones (LoRA-Att, LoRA-All) y estrategias (features visuales vs. glosas, con y sin prompt de instrucción), analizando el impacto de cada decisión de diseño sobre la calidad de la traducción.
5. **Estudiar el efecto del escalado del modelo de lenguaje.** Comparar el rendimiento de Qwen2.5-1.5B y Qwen2.5-7B bajo la misma configuración de LoRA para determinar si la capacidad representacional del modelo base es el factor limitante principal en un régimen de datos reducido como el de la lengua de signos.
6. **Cuantificar el coste computacional de cada configuración.** Medir el número de parámetros entrenables y el consumo de memoria GPU de cada variante experimental, evaluando la relación rendimiento-coste de cada aproximación de cara a su viabilidad en escenarios con recursos limitados.

1.3. Estructura del documento

La memoria se organiza en los siguientes capítulos. El Capítulo 2 revisa el estado del arte en traducción automática de lengua de signos, desde los paradigmas históricos hasta los enfoques más recientes basados en modelos de lenguaje, y presenta los conjuntos de datos relevantes. El Capítulo 3 expone los fundamentos teóricos necesarios para comprender el trabajo: mecanismos de atención, *transformers*, CTC y LoRA. El Capítulo 4 detalla las decisiones de diseño adoptadas: el estimador de pose, el proceso de normalización, la arquitectura MSKA y la integración del SLM mediante prefijo visual. El Capítulo 5 presenta los experimentos realizados y analiza los resultados, incluyendo el estudio de ablación y la comparativa global. Finalmente, el Capítulo 6 recoge las conclusiones del trabajo y propone líneas de investigación futura.

Capítulo 2

Estado del Arte

En este capítulo analizaremos el contexto general de la traducción de lengua de signos, desde su evolución hasta la actualidad. Finalmente, estudiaremos el estado del arte específico a los modelos basados en *keypoints* y presentaremos los conjuntos de datos relevantes.

2.1. Evolución de la traducción de lengua de signos

Como vimos en la introducción, la traducción automática de la lengua de signos es un problema complejo que implica convertir una secuencia de vídeo continuo en una secuencia de texto en un lenguaje hablado, superando grandes barreras espaciales, sintácticas y gramaticales. A lo largo de los últimos años ha habido grandes avances en las técnicas utilizadas. De similar forma a las tareas de traducción tradicionales, se han ido preferido arquitecturas basadas en mecanismos de atención, *vision transformers* o incluso *LLM*.

Históricamente, los primeros sistemas de reconocimiento y traducción dependían en gran medida de *hardware* intrusivo, como guantes de datos o sensores de movimiento, y de enfoques de traducción automática estadística que operaban bajo reglas rígidas elaboradas manualmente por lingüistas. Estos métodos, aunque pioneros en su momento, presentaban una escalabilidad muy limitada y fracasaban al intentar capturar la naturaleza continua, fluida y multimodal de la lengua de signos en entornos reales.

Con el auge del aprendizaje profundo, el paradigma cambió hacia el uso de Redes Neuronales Convolucionales (*CNN*) acopladas a modelos secuenciales como los Modelos Ocultos de Markov (*HMM*) o las Redes Neuronales Recurrentes (*RNN/LSTM*). Estas arquitecturas permitieron, por primera vez, extraer características espaciales directamente de los píxeles del vídeo y modelar la dependencia temporal de los signos de forma mucho más robusta. Sin embargo, la traducción completa hacia texto hablado seguía siendo deficiente debido al problema de pérdida de memoria a largo plazo en secuencias extensas

y a la dificultad computacional de alinear correctamente cientos de fotogramas con unas pocas palabras.

La llegada de los modelos secuencia a secuencia (*Seq2Seq*) y, especialmente, de la arquitectura *Transformer*, cambió de forma significativa el rumbo de la investigación en este campo. Uno de los trabajos más influyentes fue el de Camgoz et al. [1], que mostró que la traducción de la lengua de signos podía tratarse con éxito como una tarea de Traducción Automática Neuronal (NMT).

Desde entonces el enfoque parece haberse desplazado de sistemas que intentaban alinear información fotograma a fotograma, al uso de *Vision Transformers* (ViT) para lograr una comprensión espacial global más rica.

Más recientemente, la tendencia dominante ha sido la integración de Modelos de Lenguaje Grande (LLM) que actúan como potentes correctores semánticos y generadores de texto fluido, abriendo la puerta a sistemas híbridos capaces de interpretar el contexto global mucho más allá de la simple detección de gesticulaciones aisladas.

2.2. Modelos basados en glosas (dos etapas)

Durante años, el enfoque predominante para abordar la traducción de la lengua de signos ha sido dividir el problema en dos tareas secuenciales e independientes. Este paradigma, conocido como enfoque basado en glosas o de dos etapas, utiliza las **glosas** (etiquetas textuales que representan el significado léxico básico de un signo) como representación intermedia.

El procesamiento se estructura de la siguiente manera:

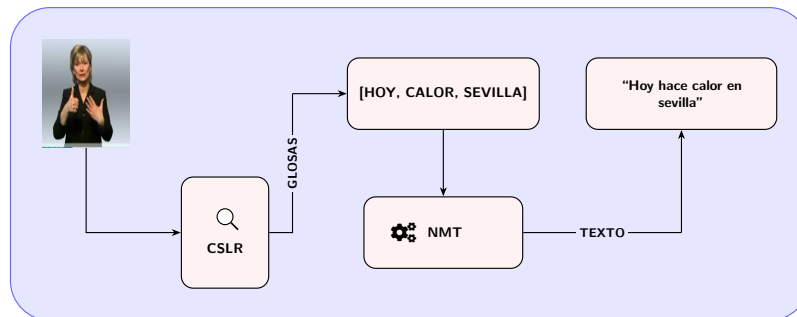


Figura 2.1: Esquema de la arquitectura basada en glosas

1. **Reconocimiento Continuo de la Lengua de Signos (CSLR):** Un modelo de visión por computador procesa la secuencia de vídeo y genera una secuencia de glosas.
2. **Traducción Automática Neuronal (NMT):** Un modelo de procesamiento de lenguaje natural toma esa secuencia de glosas (que carece de conjugaciones y sigue

la gramática espacial del lenguaje de signos) y la traduce a una oración gramaticalmente correcta en un idioma hablado.

2.2.1. Ventajas del enfoque

La principal ventaja de este método es la simplificación de la tarea conjunta. Identificar directamente representaciones de vídeo de alta dimensionalidad a texto hablado es un problema de optimización sumamente complejo, agravado por la escasez histórica de conjuntos de datos masivos que contengan pares directos de vídeo y texto. Al introducir las glosas como un ancla intermedia, los investigadores pudieron entrenar redes más estables y aprovechar las arquitecturas NMT ya existentes para la segunda etapa.

Un ejemplo destacado de los avances logrados en la primera etapa es el modelo **STMC** (*Spatial-Temporal Multi-Cue*) propuesto por Zhou et al. [2] en 2020. Aunque investigaciones previas ya reconocían el carácter multimodal de la lengua de signos, STMC fue pionero en procesar de forma conjunta diferentes señales visuales (forma de las manos, expresiones faciales y postura corporal) dentro de una única red neuronal entrenable de principio a fin, reduciendo significativamente la tasa de error en el reconocimiento continuo de secuencias de glosas.

2.2.2. Desventajas: propagación de errores

A pesar de su éxito inicial y de su interpretabilidad (al poder visualizar qué glosas ha detectado el modelo), el paradigma de dos etapas sufre de un cuello de botella por la **propagación de errores**.

En este esquema arquitectónico, los dos módulos actúan de forma aislada. Si el modelo de visión comete un error, omite un signo por oclusión, o predice una glosa equivocada, esta información errónea se transfiere directamente al modelo de lenguaje. Al no tener acceso a la secuencia de vídeo original para extraer contexto visual complementario, el modelo de traducción, por muy avanzado que sea, intentará traducir una premisa falsa, resultando en una oración carente de sentido.

A este problema se suma el coste prohibitivo de anotación de datos. Entrenar el primer módulo requiere bases de datos etiquetadas a nivel de glosa, un proceso manual, lento y costoso que debe ser realizado por expertos lingüistas, lo que restringe severamente el volumen de datos de entrenamiento disponibles. Aunque ha habido muchos esfuerzos para solventar estas limitaciones, como la generación de glosas de forma automática utilizado, por ejemplo, LLMs [18], este problema no está cerca de estar resuelto.

Estos factores y la disponibilidad de equipos de mayor potencia obligó y permitió a la comunidad investigadora a replantearse la arquitectura fundamental de los sistemas, impulsando la búsqueda de alternativas que mitigaran esta dependencia estricta de las glosas intermedias.

2.3. Modelos libre de glosas (una etapa)

Para sortear el cuello de botella de la propagación de errores y eliminar la necesidad de costosas anotaciones a nivel de glosa, se exploró el paradigma de traducción de principio a fin (*end-to-end*), comúnmente denominado *Gloss-Free SLT*.

Estos modelos proponen asociar directamente la secuencia continua de vídeo a la secuencia de texto hablado, unificando la extracción de características espacio-temporales y la generación de lenguaje en una única arquitectura conjunta. Al prescindir de la representación intermedia explícita, el modelo se ve forzado a aprender de manera implícita las alineaciones entre el espacio visual de los signos y el espacio semántico del texto.

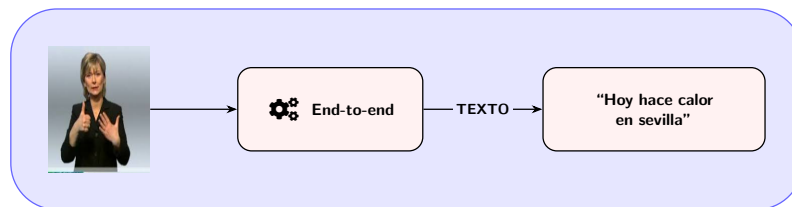


Figura 2.2: Esquema de la arquitectura basada en modelos de una etapa

Enfoques recientes en esta línea han adoptado arquitecturas basadas en *transformers* visuales y de texto, logrando resultados prometedores al permitir que el modelo atienda dinámicamente a toda la secuencia de vídeo original durante el proceso de decodificación.

2.3.1. El límite de la traducción directa

Sin embargo, el paradigma *end-to-end*, como es común, se ha topado con la barrera de la escasez de datos. Mientras que la traducción automática tradicional entre lenguajes hablados se entrena con cientos de millones de pares de oraciones, los conjuntos de datos de traducción de lengua de signos apenas superan las decenas de miles de ejemplos.

Sin la estructura y el anclaje semántico que proporcionan las glosas intermedias, los modelos libres de glosas sufren para aprender la compleja correspondencia multimodal desde cero, lo que provoca que su rendimiento se estanque.

En mi opinión esto es evidencia de que descartar por completo las glosas no era la solución definitiva, sino que el verdadero desafío residía en encontrar una manera de utilizarlas siendo tolerantes a sus imperfecciones. Esto sentó las bases para el siguiente gran salto tecnológico: el uso de Modelos de Lenguaje (*LLMs*) para el razonamiento sobre secuencias ruidosas.

2.4. El renacimiento de las glosas mediante Modelos de Lenguaje (LLMs)

Como se concluyó en la sección anterior, la dificultad de entrenar modelos directamente de vídeo a texto (*end-to-end*) devolvió el interés al uso de representaciones intermedias. Sin embargo, para no caer nuevamente en el problema de la propagación de errores, se comenzó a explorar las capacidades emergentes de los Modelos de Lenguaje Grande (LLMs).

Los LLMs, preentrenados con cantidades **masivas** de texto, poseen una capacidad de razonamiento e inferencia de contexto sin precedentes. Esto los hace candidatos ideales para recibir secuencias de glosas ruidosas (con omisiones, repeticiones o errores de detección) y reconstruir el significado lógico de la oración, mitigando la fragilidad de los enfoques NMT clásicos.

2.4.1. Traducción híbrida: el enfoque Spotter+GPT

Un hito reciente que ilustra este cambio de paradigma es el trabajo propuesto por Sincan et al. [20] en 2024. Su método, denominado **Spotter+GPT**, innova al prescindir por completo del entrenamiento de un modelo de traducción específico de glosa a texto.

A diferencia de los enfoques convencionales que presentan un bajo rendimiento en vocabularios extensos, Spotter+GPT propone una arquitectura híbrida que combina el reconocimiento visual clásico con las capacidades de razonamiento *zero-shot* (sin entrenamiento previo específico) de un LLM comercial.

La arquitectura del sistema se divide en dos componentes principales:

- **Módulo de Detección Visual (*Sign Spotter*):** Utiliza un modelo I3D (basado en características espacio-temporales densas) entrenado con un gran conjunto de datos para identificar signos individuales. Este módulo procesa el vídeo continuo mediante una técnica de ventana deslizante para extraer predicciones de glosas, aplicando posteriormente un filtrado basado en umbrales de confianza para consolidar la secuencia temporal.
- **Módulo de Generación de Lenguaje (LLM):** Emplea el modelo *gpt-3.5-turbo* de OpenAI para recibir de forma directa la secuencia de glosas detectadas. Mediante ingeniería de instrucciones, el modelo transforma esta lista en una oración coherente en el lenguaje hablado de destino, superando de forma dinámica las diferencias en el orden gramatical.

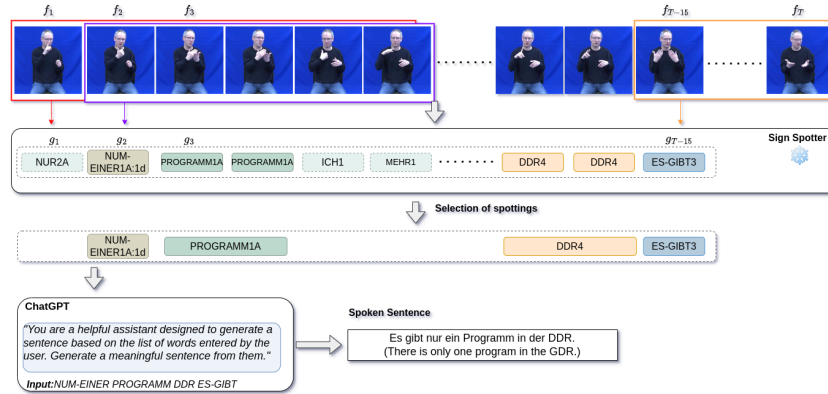


Figura 2.3: Estructura del modelo Spotter+GPT propuesto por Sincan et al. [20]

Los experimentos realizados demostraron que Spotter+GPT supera a los modelos secuenciales dedicados en métricas de calidad y similitud semántica, como BLEURT [25]¹. Además, el enfoque destaca cualitativamente por su gran resiliencia: el LLM logra mantener la coherencia semántica en la oración hablada final incluso cuando el detector visual pierde información o introduce glosas adicionales incorrectas.

2.4.2. Limitaciones de los LLMs masivos en SLT

Si bien enfoques como Spotter+GPT han demostrado que los grandes modelos de lenguaje pueden solucionar el cuello de botella histórico de las glosas, introducen nuevas barreras para su despliegue en escenarios reales.

En primer lugar, la dependencia de modelos masivos comerciales (como GPT-3.5 o GPT-4) requiere conexión constante a internet e incurre en costes recurrentes por uso de API. En segundo lugar, enviar transcripciones o datos derivados de usuarios a servidores externos plantea serios problemas de privacidad, un aspecto crítico en aplicaciones de accesibilidad. Finalmente, la combinación de extractores visuales pesados (como I3D) operando sobre los píxeles del vídeo junto con la latencia de red de un LLM en la nube, hace que la traducción en tiempo real en dispositivos de bajo consumo sea inabarcable.

2.5. Modelos basados en esqueletos (*keypoints*)

Como hemos observado, el procesamiento de secuencias de vídeo RGB en la traducción de lengua de signos conlleva un alto coste computacional y una gran vulnerabilidad frente a las fluctuaciones del fondo, la iluminación o la apariencia del signante. Para mitigar estos problemas, una de las ramas de la investigación ha impulsado el uso de *keypoints* como representación visual de entrada.

¹BLEURT es una métrica de evaluación automática basada en BERT que mide la calidad de textos generados por IA evaluando su similitud semántica con un texto de referencia.

El uso de *keypoints* extraídos mediante estimadores de pose (como HRNet, OpenPose) no solo reduce sustancialmente los requisitos computacionales al transformar tensores de vídeo masivos en matrices ligeras de coordenadas, sino que también elimina los sesgos visuales del entorno. Además, como se menciona en la introducción, este enfoque aborda las crecientes preocupaciones sobre la privacidad, ya que anonimiza al usuario al prescindir de sus rasgos biométricos faciales o de su entorno personal.

2.5.1. Atención multiescala: El modelo MSKA

Para superar las limitaciones topológicas de los modelos basados en grafos, Guan et al. [19] propusieron en 2024 la red de atención multiescala/multiflujo basada en *keypoints* (**MSKA**, *Multi-Stream Keypoint Attention*). Esta arquitectura se inspira en la cognición humana, que interpreta la lengua de signos analizando la interacción dinámica entre la configuración de las manos y los gestos corporales adicionales.

A diferencia de sus predecesores, MSKA prescinde de topologías predefinidas y desacopla la entrada en flujos independientes (manos, rostro y cuerpo), procesándolos mediante módulos de atención pura para luego fusionarlos. Aunque los detalles arquitectónicos de este modelo se abordarán en profundidad en el capítulo de Metodología 4, es crucial destacar su enfoque para la traducción. Para conformar su sistema completo (MSKA-SLT), los autores acoplaron este extractor visual al modelo de lenguaje **mBART** [30].

Esta dependencia de una arquitectura de traducción clásica (*Seq2Seq*) presenta una limitación teórica fundamental: la **rigidez estructural**. Si el módulo visual comete errores predictivos (omisiones o glosas incorrectas debido a oclusiones temporales), modelos dedicados como mBART carecen de la capacidad de inferencia deductiva profunda necesaria para reconstruir el contexto lógico de forma autónoma.

Es precisamente este cuello de botella semántico el que motivan mi exploración de Modelos de Lenguaje Pequeños (SLM) como alternativa. La integración de un SLM promete aportar la capacidad de comprensión y corrección contextual propia de los grandes modelos generativos, manteniendo un tamaño que permita su ejecución local para garantizar la privacidad. Sin embargo, sustituir un decodificador *Seq2Seq* altamente especializado en traducción por un modelo causal de propósito general introduce nuevos y complejos desafíos de alineación y evaluación empírica. Explorar la viabilidad, el rendimiento y los retos de esta transición arquitectónica constituye el objetivo central de nuestro trabajo.

2.6. Conjuntos de datos

La evolución de las arquitecturas de traducción ha estado intrínsecamente ligada a la disponibilidad, volumen y calidad de los datos. Si comparamos el paradigma actual con el de la década pasada, la mejora es innegable, aunque los volúmenes siguen siendo notablemente inferiores en comparación con los inmensos corpus utilizados en el procesamiento de lenguaje natural tradicional.

Históricamente, los conjuntos de datos de lengua de signos se construían en entornos de laboratorio altamente controlados: fondos estáticos, iluminación impecable y dominios léxicos muy limitados (por ejemplo, vocabulario médico básico). Hoy en día, el paradigma ha virado hacia la creación de corpus en escenarios reales (*in the wild*), abordando vocabularios sin restricciones y enriqueciendo los datos con anotaciones multimodales, como *keypoints* 2D/3D preextraídos.

A continuación, exploramos los conjuntos de datos más relevantes que han marcado el progreso en este campo:

- **PHOENIX-2014T [14]:** Considerado el estándar de oro en la evaluación de modelos SLT. Consiste en grabaciones de pronósticos meteorológicos de la televisión pública alemana en Lengua de Signos Alemana (DGS). Aunque su dominio es restringido y repetitivo, su impecable alineación paralela entre vídeo, secuencias de glosas y texto hablado lo convierte en el punto de referencia ineludible de la industria. Sin embargo, cabe destacar que los vídeos originales presentan una resolución muy baja, lo que supone un desafío histórico y una limitación importante para los extractores visuales modernos.
- **CSL-Daily [15]:** Un extenso conjunto de datos de Lengua de Signos China (CSL) grabado en estudio. Destaca por alejarse de vocabularios técnicos y centrarse en escenarios de la vida cotidiana (familia, salud, escuela), ofreciendo una mayor riqueza semántica y gramatical para evaluar la generalización de los modelos en diálogos habituales. Sin embargo, presenta una notable barrera de accesibilidad ya que su distribución está estrictamente limitada a universidades e institutos con fines exclusivos de investigación. Para obtener los recursos, se exige un acuerdo formal de liberación de datos firmado obligatoriamente por personal académico a tiempo completo, **excluyendo explícitamente el acceso directo a estudiantes**.
- **How2Sign [16]:** Este corpus masivo de Lengua de Signos Americana (ASL) ejemplifica el paradigma moderno. Basado en vídeos instructivos extraídos de internet, presenta grabaciones en entornos reales con múltiples ángulos y un vocabulario extremadamente amplio y continuo, suponiendo un reto mayúsculo de generalización para los modelos actuales.
- **Sign4All [17]:** Un conjunto de datos reciente que destaca por su enfoque en la inclusión y la captura de alta calidad. La generación de los datos contó con 8 participantes grabados a una resolución de 2560×1440 y 30 fps, suponiendo un salto de calidad respecto a corpus clásicos. El conjunto original consta de 7,756 muestras de vídeo segmentadas manualmente junto con *keypoints*, ampliado mediante técnicas de aumento de datos a 61,409 muestras para respaldar el entrenamiento de arquitecturas de aprendizaje profundo. Al igual que CSL-Daily, y debido a preocupaciones sobre la privacidad de los participantes, el acceso a este *dataset* está restringido y requiere la solicitud de un Acuerdo de Uso de Datos (DUA), aunque estos permiten el uso a estudiantes.. Debido a su reciente introducción **23 de febrero de 2026** y a sus prometedoras características técnicas, su integración y su calidad se contemplan como una sólida línea de investigación para el trabajo futuro de este proyecto.

2.6.1. Selección del conjunto de datos

Para el desarrollo y validación empírica de esta investigación, se ha seleccionado el conjunto de datos **PHOENIX-2014T** como base principal para los experimentos. Existen principalmente dos razones:

1. **Comparabilidad histórica:** Al ser el estándar de oro unánime en la investigación de SLT, utilizar PHOENIX garantiza que los resultados obtenidos en este trabajo puedan ser medidos y comparados directamente con casi una década de literatura científica, proporcionando un marco de evaluación robusto.
2. **Alineación con el modelo base y accesibilidad:** PHOENIX-2014T es el corpus principal sobre el que se evaluó y validó la arquitectura MSKA original. Utilizar el mismo conjunto de datos nos proporciona una base de referencia exacta para aislar y medir el impacto real de sustituir mBART por un SLM. Además, a diferencia de corpus como CSL-Daily, que presentan fuertes barreras burocráticas de acceso, PHOENIX es de acceso público, lo que garantiza la reproducibilidad de nuestros experimentos.

Cabe hacer una mención técnica especial al conjunto de datos **Sign4All**. Si este corpus hubiera estado publicado y consolidado en el momento de la concepción y el diseño inicial de este proyecto, habría sido la opción elegida sin lugar a dudas. Su altísima resolución y su enfoque nativo en la extracción de coordenadas esqueléticas lo convierten en el entorno ideal para modelos basados en *keypoints*.

Capítulo 3

Fundamentos

3.1. Medidas de precisión en traducción

La evaluación de sistemas de traducción automática no admite una única respuesta correcta: distintas formulaciones válidas del mismo contenido pueden diferir léxicamente. Por ello se emplean métricas automáticas que aproximan el juicio humano de forma escalable.

3.1.1. WER

La **Tasa de Error de Palabras** (*Word Error Rate*, WER) mide la distancia de edición mínima a nivel de palabra entre la hipótesis del modelo y la referencia, normalizada por la longitud de esta última:

$$\text{WER} = \frac{S + D + I}{N},$$

donde S , D e I son el número de sustituciones, eliminaciones e inserciones respectivamente, y N es el total de palabras en la referencia. Es la métrica estándar para el reconocimiento de glosas, dado que penaliza directamente los errores de segmentación temporal propios de la tarea.

3.1.2. BLEU

BLEU (*BiLingual Evaluation Understudy*) [23] evalúa la calidad de la traducción midiendo la precisión de n -gramas entre el texto generado y la referencia. Para evitar que repetir palabras correctas infle artificialmente la puntuación, utiliza una precisión recortada (*clipped*):

$$p_n = \frac{\sum_C \sum_{n\text{-grama} \in C} \text{Count}_{\text{clip}}(n\text{-grama})}{\sum_C \sum_{n\text{-grama} \in C} \text{Count}(n\text{-grama})}.$$

La puntuación final incorpora una penalización por brevedad BP que descuenta traducciones más cortas que la referencia:

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right), \quad BP = \begin{cases} 1 & c > r \\ e^{1-r/c} & c \leq r \end{cases}$$

donde c y r son las longitudes totales del texto generado y de referencia. Su principal limitación es la ausencia de comprensión semántica: sinónimos o paráfrasis válidas son penalizados igual que errores reales. Se reportan BLEU-1 a BLEU-4 ($N = 1, \dots, 4$) siguiendo la convención habitual en traducción de lengua de signos.

3.1.3. ROUGE-L

ROUGE-L [24] evalúa la similitud mediante la subsecuencia común más larga (LCS) entre el texto generado y la referencia, combinando exhaustividad y precisión en una puntuación F_1 :

$$R = \frac{|\text{LCS}|}{|\text{ref}|}, \quad P = \frac{|\text{LCS}|}{|\text{hyp}|}, \quad F_1 = \frac{2RP}{R + P}.$$

A diferencia de BLEU, no exige que los n -gramas sean contiguos, lo que la hace más tolerante a pequeñas reordenaciones. Comparte con BLEU la limitación de basarse en coincidencia léxica exacta.

3.1.4. BLEURT

BLEURT (*Bilingual Evaluation Understudy with Representations from Transformers*) [25] es una métrica neuronal entrenada para correlacionar con el juicio humano. A diferencia de BLEU y ROUGE, no compara n -gramas sino representaciones semánticas obtenidas a partir de un modelo BERT preentrenado y ajustado sobre pares de frases con puntuaciones humanas. Esto le permite valorar correctamente sinónimos y paráfrasis que las métricas léxicas penalizarían. Se emplea aquí exclusivamente en el experimento *zero-shot* (Sección ??), donde la formulación libre de las traducciones hace que BLEU resulte poco informativo.

3.2. Transformers

La arquitectura **Transformer**, introducida por Vaswani et al. [11], supuso un cambio de paradigma absoluto en el procesamiento de secuencias, desplazando a las redes neuronales recurrentes. Su relevancia para el presente trabajo radica en su capacidad para gestionar dependencias de largo alcance mediante el mecanismo de autoatención (*self-attention*). Esto permite que cada elemento de una entrada (ya sea un *token* de texto o un *keypoint* del cuerpo) sea procesado en relación con todos los demás de forma paralela.

A continuación, repasaremos sus fundamentos estructurales para entender su aplicación en nuestro modelo.

3.2.1. Mecanismo de atención

La función de atención permite que cada elemento de una secuencia se contextualice con el resto de forma paralela, sin depender de un orden estrictamente secuencial. Dadas una consulta Q , unas claves K y unos valores V , la variante más extendida es la *Scaled Dot-Product Attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

donde el factor $\sqrt{d_k}$ evita que los productos escalares saturen la *softmax* y degraden los gradientes en dimensiones altas.

Para capturar simultáneamente distintos tipos de dependencias, se proyectan Q , K y V en h subespacios independientes mediante la **atención multicabeza** (*Multi-Head Attention*):

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O, \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V). \end{aligned}$$

En nuestro sistema, este mecanismo se aplica sobre secuencias de *keypoints* corporales, donde cada cabeza puede especializarse en relaciones espaciales distintas (por ejemplo, la coordinación entre manos o la relación entre postura y expresión facial).

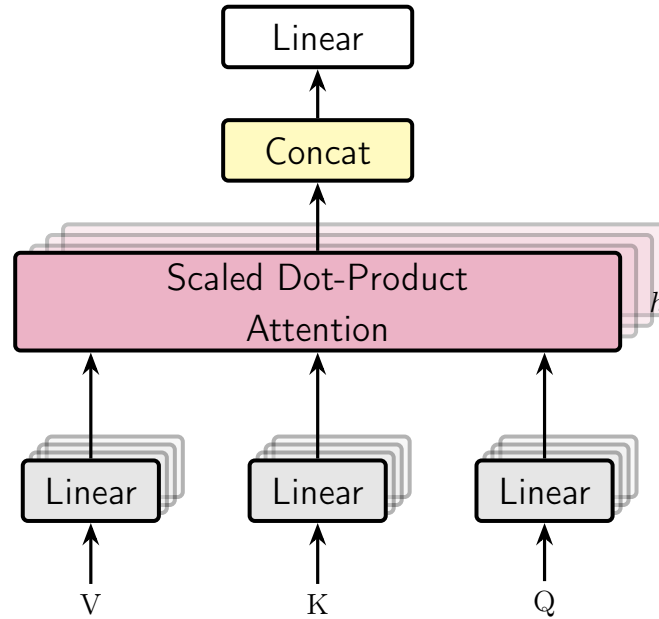


Figura 3.1: Diagrama de la atención multi-cabeza

3.2.2. Codificación posicional

Dado que la atención procesa todos los elementos en paralelo, el modelo carece de noción de orden por sí solo. La codificación posicional añade esta información sumando

a cada elemento un vector que depende de su posición p en la secuencia:

$$PE(p, 2i) = \sin\left(\frac{p}{10000^{2i/C}}\right),$$

$$PE(p, 2i + 1) = \cos\left(\frac{p}{10000^{2i/C}}\right),$$

donde i indexa la dimensión del vector y C es el tamaño del canal de entrada. El uso de frecuencias distintas permite representar posiciones absolutas y, al mismo tiempo, capturar relaciones relativas entre elementos distantes. En el contexto de datos esqueléticos, esta codificación se adapta para separar la dimensión espacial (identidad de cada articulación) de la temporal (instante del fotograma), como se detalla en la Sección 3.4.

3.3. Clasificación Temporal Conexionista

La *Connectionist Temporal Classification* (CTC) es un método que surge para entrenar Redes Neuronales Recurrentes en el etiquetado de secuencias continuas, eliminando la necesidad de utilizar datos de entrenamiento presegmentados o alineados manualmente.

Esto es especialmente importante en tareas perceptuales y de visión por computador, donde la secuencia de entrada, por ejemplo, una serie de fotogramas de vídeo que capturan a un signante, es habitualmente más larga que la secuencia objetivo de glosas o etiquetas a predecir. Al ser de longitudes distintas y carecer de una segmentación temporal explícita, no existe una correspondencia o alineación a priori entre ambas secuencias.

Debido a la alta carga matemática de este concepto, nos limitaremos a explicar el funcionamiento y la idea que propone a grandes rasgos sin entrar en la carga teórica pues no es relevante a este trabajo, aún así todo el formalismo teórico se encuentra en *Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks* [26].

Para resolver este problema de alineación, la intuición principal de la CTC radica en interpretar las salidas de la red neuronal (la secuencia completa) como una distribución de probabilidad sobre todas las posibles **secuencias de etiquetas**, condicionadas a la secuencia de entrada. Combinan dos técnicas principalmente.

Blank Label

Para cada fotograma individual (cada instante de tiempo), a la capa softmax que nos devolvía una distribución de probabilidad sobre todas las glosas posibles del vocabulario, se extiende con un token especial “BLANK”. Este token representa la probabilidad de no observar ninguna etiqueta en ese instante de tiempo específico.

Mapping

La red genera predicciones para cada fotograma individual, pero la CTC aplica una regla de transformación para convertir esa ruta temporal extendida en una secuencia final

coherente. Esta regla consiste en eliminar primero todas las etiquetas que se repiten de forma consecutiva y, posteriormente, suprimir todos los tokens en blanco.

De forma intuitiva, esto significa que el modelo registrará una nueva etiqueta únicamente cuando la red pase de predecir un espacio en blanco a predecir una glosa, o cuando cambie directamente de una glosa a otra distinta.

Llevado a la traducción de lengua de signos, si un gesto dura varios fotogramas, una idea del proceso es la siguiente.

$$\underbrace{[-, -, \text{HOLA}, \text{HOLA}, \text{HOLA}, -, -]}_{7 \text{ fotogramas}} \rightarrow \underbrace{[\text{HOLA}]}_{\text{Una sola glosa}}$$

Finalmente, durante la fase de entrenamiento, la CTC no penaliza a la red por no adivinar el momento exacto en el que empieza o termina un signo. En su lugar, CTC le da ignora exactamente cuándo ocurre el signo, acepta que hay muchas formas válidas de acertar a lo largo del tiempo. Para un vídeo de 5 frames, todas estas “rutas” de la red neuronal son válidas porque, al aplicar la regla de la CTC, todas colapsan en el mismo resultado final, [HOLA].

- **Ruta A:** [GRACIAS, GRACIAS, -, -, -]
- **Ruta B:** [-, GRACIAS, GRACIAS, -, -]
- **Ruta C:** [-, -, -, GRACIAS, GRACIAS]

Durante el entrenamiento, no se intenta forzar que ocurra solo una ruta que se considere “perfecta”. Lo que se hace es sumar la probabilidad de cada ruta posible que devuelva el mismo resultado tras aplicar las reglas. Al maximizar esta suma, la red se entrena para predecir la etiqueta correcta sin atarse a una plantilla de tiempo rígida.

En su lugar, utiliza una función objetivo (CTCloss) que suma las probabilidades de todas las alineaciones válidas posibles, maximizando de forma directa la probabilidad global de obtener la secuencia de glosas correcta. Esto permite que la red aprenda automáticamente de forma discriminativa y se vuelva robusta ante variaciones en la velocidad o duración temporal de los signos.

Si x representa la secuencia de entrada, z es la secuencia objetivo, y $p(z|x)$ es la probabilidad total acumulada de todas las alineaciones válidas que colapsan en esa secuencia de glosas tras aplicar las reglas de remoción de blancos y repetidos. La función de pérdida CTC se define como la verosimilitud logarítmica negativa de las secuencias objetivo mapeadas.

$$O^{ML}(S, N_w) = - \sum_{(x,z) \in S} \log p(z|x)$$

3.4. Decoupled Spatial-Temporal Attention (DSTA)

Tradicionalmente, los métodos de reconocimiento de acciones (basados en redes *RNN* o *CNN*) dependían de reglas manuales o topologías de grafos prediseñadas para modelar

cómo se conectan las articulaciones del cuerpo. El problema es que estos diseños manuales no siempre son óptimos y limitan la generalización del modelo.

Para solucionar esto, **Lei Shi et al.** en [22] proponen un enfoque basado exclusivamente en mecanismos de atención inspirado en los *transformers* vistos en la sección ?? . Esto permite que la red aprenda automáticamente las dependencias espaciales y temporales entre las articulaciones sin necesidad de conocer sus posiciones exactas o conexiones anatómicas previas, además implementan una técnica de separación de datos para captar distintos tipos de movimientos.

3.4.1. Módulos de atención espacial y temporal

Si recordamos en la sección anterior de *transformers* que la entrada de datos es una secuencia bidimensional: una matriz $X \in \mathbb{R}^{N \times C}$, donde N es el número de elementos y C es el número de canales.

Sin embargo, los datos dinámicos del esqueleto tienen tres dimensiones: **espacio**, **tiempo** y **canales**. Podemos representarlos como un tensor de tercer orden: $X \in \mathbb{R}^{N \times T \times C}$, donde N es el número de articulaciones, T es el número de fotografías (frames) y C son los canales de coordenadas.

La forma ingénuo de representar los datos, ya probado por otros autores anteriores, es aplanar el espacio y el tiempo en una sola dimensión $\hat{N} = NT$, convirtiendo el tensor de tercer orden en $X \in \mathbb{R}^{\hat{N} \times C}$. Este enfoque presentaba varias desventajas, a priori, estaban tratando el espacio y el tiempo como si fueran lo mismo y compartieran estructura, lo cual físicamente no tiene sentido. Por otra parte, el coste computacional de este enfoque es muy alto en tiempo y en memoria.

Este papel propone tres métodos para este desacoplamiento. Los explicaremos brevemente utilizando como ejemplo la atención espacial.

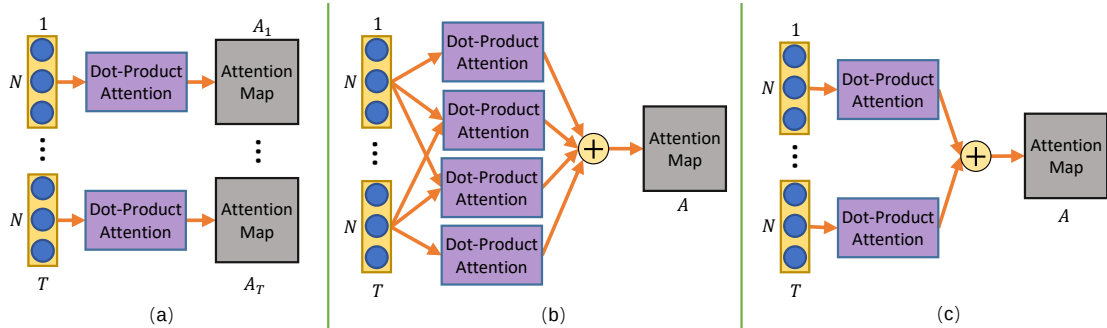


Figura 3.2: Las tres estrategias propuestas en [22]. Grafico sacado de ese papel.

Estrategia (a)

Aquí se calcula un mapa de atención único para cada fotograma de manera independiente.

$$Attention_t = softmax(\sigma(X_t)\phi(X_t)'),$$

donde, $Attention_t \in \mathbb{R}N \times N$ es el mapa de atención para el fotograma t ; $X_t \in \mathbb{R}^{N \times C}$ son los datos de ese fotograma; y, σ y ϕ denotan funciones de *embedding*.

Este método no solo tiene un alto coste computacional sino que, además, es incapaz de modelar las relaciones fuera de un mismo fotograma, carece de contexto global.

Estrategia (b)

En un giro de 180° los autores proponen otro método. En vez de restringirnos en un fotograma, calculamos las relaciones entre todas las articulaciones a lo largo de todos los fotogramas de forma simultánea. El mapa de atención resultante se comparte para todos los fotogramas.

$$Attention = softmax(\sum_t^T \sum_\tau^T (\sigma(X_t)\phi(X_\tau)')).$$

Aunque es **muy** potente, este método es aún más computacionalmente pesado.

Estrategia (c)

c La estrategia que finalmetente propone el papel consiste en solo considerar las articulaciones dentro del mismo fotograma para calcular las relaciones, pero luego sumar y compartir los mapas de atención de todos los fotogramas.

$$Attention = softmax(\sum_t^T (\sigma(X_t)\phi(X_t)')).$$

Si en lugar de tener T matrices separadas de tamaño $N \times C$ creamos una nueva matriz M de tamaño $N \times TC$ concatenando horizontalmente las matrices de cada fotograma,

$$(X_0 \cdots X_T),$$

por las propiedades de las multiplicaciones de las matrices por bloques, al realizar el producto por su traspuesta obtenemos una matriz cuadrada $N \times N$.

Esa única operación produce matemáticamente el mismo resultado exacto que si hubieramos multiplicado y sumado cada fotograma por separado. Pasamos de hacer T operaciones a hacer una sola operación (que está muy optimizada en las máquinas modernas).

3.4.2. Codificación Posicional

Como ocurre en otros modelos de atención pura, las articulaciones del esqueleto se organizan en forma de tensor para poder introducir las en las redes neuronales. Estos tensores carecen de orden o estructura que indique por sí mismo qué representa cada uno (por ejemplo, el índice de la articulación o el índice del fotograma), es necesario usar un módulo de codificación posicional que asigne un identificador único a cada articulación.

Ahora bien, en texto, las palabras van una detrás de otra (1D). Pero los datos del esqueleto tienen dos dimensiones, el espacio (las diferentes articulaciones) y el tiempo (los fotogramas).

Una estrategia ingenua de codificación unificaría estas dimensiones, asignando el valor p de forma secuencial a través de todo el tensor aplanado. Por ejemplo, con $N = 3$, en el fotograma $t = 1$, sus posiciones serían 1, 2, 3 y en el fotograma $t = 2$, serían 4, 5, 6. El problema de esta topología es que la red no puede distinguir adecuadamente la misma articulación en fotogramas diferentes, ya que la articulación $n = 1$ recibe representaciones posicionales matemáticamente dispares ($PE(1)$ vs $PE(4)$).

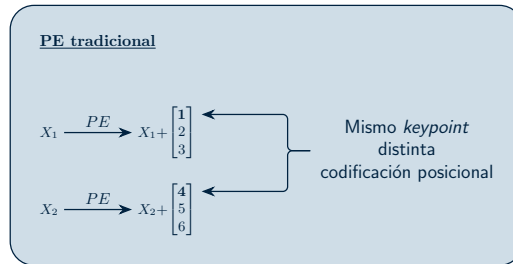


Figura 3.3: Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un **esquema** de lo que ocurre.

Le estamos añadiendo dificultad a la red a la hora de aprender que la articulación 1 y la articulación 4 son, en realidad, la misma parte del cuerpo en diferentes momentos.

Así, para preservar la semántica del esqueleto, los autores proponen separar el proceso en codificación espacial y codificación temporal.

Codificación de Posición Espacial

Las articulaciones dentro de un mismo fotograma se codifican de forma secuencial, mientras que la misma articulación en fotogramas distintos mantiene exactamente la misma codificación.

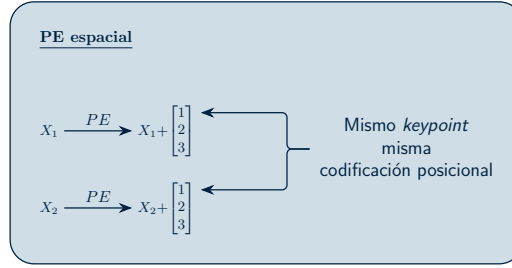


Figura 3.4: Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un **esquema** de lo que ocurre.

Utilizar esta codificación permite que, sin importar el fotograma, una articulación siempre tenga el mismo vector de identidad espacial.

Codificación de Posición Temporal

Es el proceso análogo e inverso. Las articulaciones dentro de un mismo fotograma se codifican de forma secuencial, mientras que la misma articulación en fotogramas distintos mantiene exactamente la misma codificación.

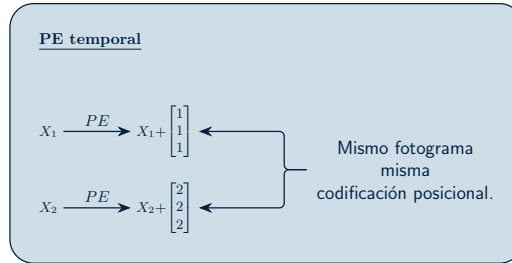


Figura 3.5: Consideramos 2 fotogramas con 3 puntos clave cada uno. Esto es un **esquema** de lo que ocurre.

3.4.3. Spatial Global Regularization (SGR)

En el procesamiento de imágenes, las redes convolucionales (CNNs) aprovechan que los píxeles cercanos forman bordes y formas, compartiendo pesos locales para encontrar patrones en cualquier parte de la imagen.

En los datos esqueléticos, tenemos un tipo diferente de ventaja, en su mayoría, la anatomía humana es **universal** y **constante**. El *keypoint* que representa la “mano derecha” tiene un significado físico y semántico específico, el cual se mantiene fijo para todos los fotogramas de un video y es consistente a lo largo de todos los sujetos en la base de datos.

Basándose en este conocimiento previo, los autores de la **DSTA** proponen esta regularización para obligar al modelo a aprender atenciones espaciales más generales y aplicables a diferentes muestras.

Para ello, se crea una matriz entrenable o mapa de atención global de tamaño $N \times N$, compartida para todas las muestras de datos. Esta matriz actúa como una plantilla que representa los patrones de relaciones que poseen las distintas articulaciones humanas entre sí, que normalmente se mantienen constantes.

Finalmente, este nuevo mapa de atención se combina con el obtenido en la Sección 3.4.1 junto con un factor α que se encarga de controlar la potencia de esta regularización.

Esta regularización se aplica única y exclusivamente a la dimensión espacial, pues la dimensión temporal no posee esta característica de alineación semántica. Forzar a la red a aprender un patrón unificado global para el tiempo carece de sentido lógico y, como demostraron empíricamente en sus estudios de ablación, perjudicaría el rendimiento del modelo.

El siguiente diagrama muestra la estructura completa del módulo de atención espacial. La atención temporal es análoga salvo por los matices mencionados.

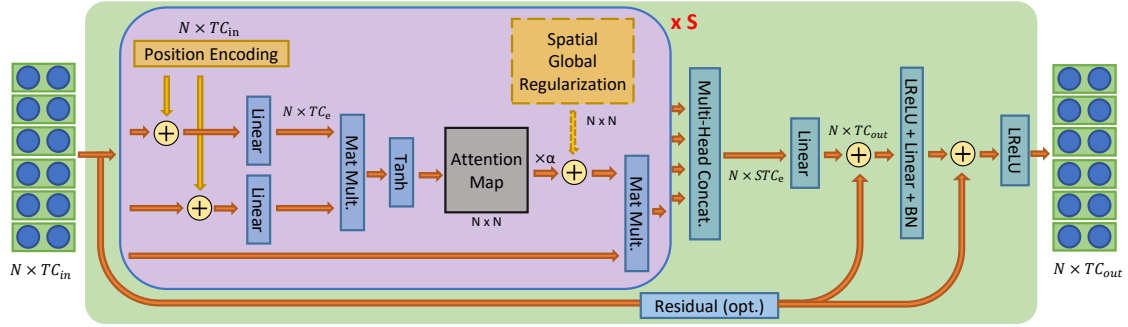


Figura 3.6: Esquema del módulo de atención **espacial**. Obtenido de [22]

Así, en resumen, a la matriz de entrada $X \in \mathbb{R}^{N \times TC_{in}}$ se le suma la codificación de posición espacial. Luego, se procesa mediante dos funciones de mapeo lineal para proyectarla a un espacio con dimensiones $N \times TC_e$. Utilizar un número de canales (C_e) menor al de salida ayuda a eliminar la redundancia de características y disminuye el coste computacional de la red.

A continuación, se calcula el mapa de atención utilizando la *estrategia* (c) y se le añade la regularización global espacial. Un detalle muy interesante es el uso de la función de activación *Tanh* en lugar de la habitual *Softmax* para generar este mapa. La razón que dan los autores de este modelo es que la función *Tanh* no restringe los valores a números positivos, lo que otorga al modelo mucha más flexibilidad al permitirle capturar y generar relaciones “negativas” entre las articulaciones. Por último, este mapa de atención resultante se multiplica matricialmente con los datos de la entrada original para obtener las características de salida finales.

Para que el modelo pueda analizar los datos desde múltiples perspectivas simultáneamente, no utiliza una sola cabeza de atención, sino un total de S cabezas independientes dentro del mismo módulo. Los resultados de todas estas cabezas se concatenan y se procesan a través de una capa lineal para proyectarlos al espacio de salida final $R^{N \times TC_{out}}$. Al igual que en el *transformer* original, el proceso termina pasando por una capa *feed-forward* que utiliza *leaky ReLU* como función de activación. Además, se incluyen dos conexiones residuales a lo largo del módulo para estabilizar el entrenamiento de la red e integrar mejor las diferentes características extraídas.

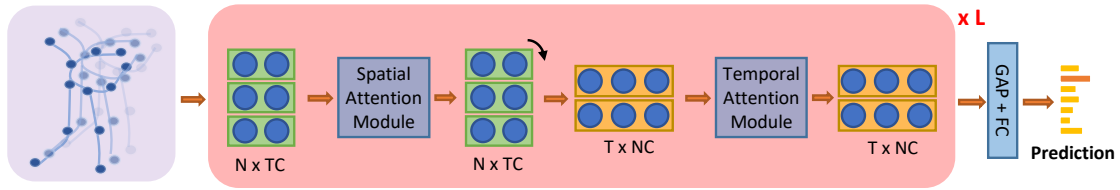


Figura 3.7: Esquema de la red. Obtenido de [22]

La arquitectura **DSTA-Net** organiza este proceso apilando un total de L capas. En cada una de estas capas, el tensor de entrada se procesa secuencialmente: primero entra al módulo de atención espacial (tratando los datos como N articulaciones) para aprender la estructura física, y luego la matriz resultante se transpone para entrar al módulo de atención temporal (tratando los datos como T fotogramas) para aprender el movimiento. Una vez que los datos han pasado por todas las capas, las características finales se reducen mediante un *Global Average Pooling* y se envían a una capa completamente conectada que generaría las puntuaciones finales de clasificación de la acción.

3.5. LoRA

El *finetuning* completo de un modelo de lenguaje grande actualiza todos sus parámetros, lo que resulta prohibitivo en memoria y cómputo a medida que el modelo crece. **LoRA** (*Low-Rank Adaptation*) [34] propone una alternativa eficiente: congelar los pesos preentrenados e inyectar actualizaciones de **rango reducido** en las capas seleccionadas.

La intuición es que, aunque los modelos preentrenados tienen millones de parámetros, los cambios necesarios para adaptarlos a una tarea concreta residen en un subespacio de dimensión mucho menor. Formalmente, dado un peso $W_0 \in \mathbb{R}^{d \times k}$, LoRA descompone su actualización como el producto de dos matrices de rango bajo:

$$W = W_0 + \Delta W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k).$$

Durante el entrenamiento, W_0 permanece congelado y solo se actualizan A y B , reduciendo drásticamente el número de parámetros entrenables. A se inicializa con una distribución gaussiana y B a cero, de modo que $\Delta W = 0$ al inicio y el modelo parte del comportamiento preentrenado.

La escala de la actualización se controla mediante un hiperparámetro α , de forma que la contribución efectiva de los adaptadores es:

$$W = W_0 + \frac{\alpha}{r} BA.$$

El cociente α/r actúa como un factor de escala global: aumentar α amplifica la influencia de los adaptadores, mientras que aumentar r incrementa la capacidad expresiva de ΔW a costa de más parámetros entrenables. En la práctica, fijar $\alpha = 2r$ es una elección habitual que mantiene las actualizaciones en una escala comparable a los pesos originales.

LoRA se aplica típicamente sobre las proyecciones de atención (W_Q, W_K, W_V, W_O), aunque puede extenderse a las capas *feed-forward* del bloque transformer cuando se requiere mayor capacidad de adaptación. La elección de qué capas adaptar supone un compromiso entre rendimiento y coste computacional que se analiza empíricamente en la Sección ??.

Capítulo 4

Metodología

En este capítulo se detallan las decisiones de diseño y las técnicas experimentales implementadas para el desarrollo del sistema de traducción. En primer lugar, se justificará el método seleccionado para la extracción visual de características. Posteriormente, se describirá el proceso de preprocesado y normalización de los datos espaciales. Finalmente, se desglosará la arquitectura base de los modelos.

4.1. Extracción de *Keypoints*

Extraer *keypoints* significa, en esencia, traducir el cuerpo humano a un mapa estructurado de puntos anatómicos. Al capturar imágenes o vídeo, el sistema localiza coordenadas bidimensionales que representan articulaciones, extremidades y rasgos faciales. Esta representación abstrae la información visual relevante para el movimiento y la postura, descartando ruido visual irrelevante como el fondo, la iluminación o la apariencia superficial. Es una solución compacta, generalizable y, además, respetuosa con la privacidad.

En los últimos años, el avance conjunto de la visión por computador y el aprendizaje profundo ha dado lugar a múltiples herramientas de estimación de pose. La traducción de lengua de signos exige capturar movimientos veloces y sutiles, asegurar la coherencia temporal de las trayectorias a lo largo del vídeo y operar bajo un coste computacional que viabilice futuras aplicaciones de traducción en tiempo real.

Al mirar el estado del arte actual, las herramientas de referencia muestran flaquezas de cara a este dominio. Un sistema clásico como **OpenPose** [9], pionero en la estimación multi-persona, se ve superado hoy en precisión y arrastra un coste de cómputo excesivo para flujos de trabajo escalables, registrando un *F1-Score* del 83,67% en el conjunto de datos estándar MPII.

Por otro lado, la popular opción de Google, **MediaPipe Holistic** [8], destaca por su velocidad, no obstante, su diseño está fuertemente optimizado para el seguimiento de

un único individuo en primer plano. Cuando el fondo se vuelve caótico o cruzan terceras personas, el modelo puede perder el foco sobre el signante principal o introducir ruido en las estimaciones anatómicas.

Una alternativa con una precisión espacial notable en el análisis estático es **MMPose** (parte del proyecto OpenMMLab), que alcanza un *F1-Score* del 87,12% gracias a su diseño arquitectónico, orientado a preservar representaciones de alta resolución. Además, por defecto, procesa los fotogramas de manera aislada. Sin un hilo conductor temporal implícito, el resultado suele sufrir de temblores constantes (*jittering*). En lengua de signos, donde la trayectoria del movimiento es tan elocuente como la propia postura de las manos, estas oscilaciones artificiales distorsionan el mensaje original.

4.1.1. El ecosistema MMPose y la adopción de AlphaPose

Al evaluar el panorama actual de la estimación de pose, es imperativo reconocer la transición de la industria hacia librerías modulares como **MMPose**. A diferencia de los *frameworks* monolíticos, MMPose se ha consolidado como el estándar moderno gracias a su extrema versatilidad. Permite intercambiar arquitecturas (*backbones*), cabezales de detección y funciones de pérdida con facilidad, integrando casi de inmediato los modelos más recientes y eficientes del estado del arte, como RTMPose.

Frente a este ecosistema tan dinámico, **AlphaPose** [7] representa una herramienta pionera con una arquitectura mucho más enfocada y cerrada. No obstante, utilizaremos AlphaPose en este trabajo por dos motivos. En primer lugar, una inercia histórica, AlphaPose ha sido la base de numerosas investigaciones previas, lo que garantiza un entorno probado, maduro y directamente comparable con la literatura existente. En segundo lugar, y más importante, su diseño integral “listo para usar” incorpora nativamente soluciones específicas para el seguimiento de personas en vídeo, algo que en librerías más modulares requiere un ensamblaje ligeramete más manual.

4.1.2. Configuración y arquitectura de AlphaPose

El primer acierto de esta herramienta radica en cómo gestiona el espacio cuando hay más de una persona en escena. AlphaPose utiliza una estrategia descendente (*Top-Down*) basada en la arquitectura RMPE (*Regional Multi-Person Pose Estimation*). A diferencia de los enfoques ascendentes (*Bottom-Up*), que localizan todas las articulaciones de la imagen a la vez para luego intentar agruparlas por individuos, RMPE emplea primero un detector de objetos. Acota a cada sujeto en una caja delimitadora (*bounding box*) y, solo entonces, calcula su postura por separado. De este modo, el signante principal queda blindado frente al ruido del entorno o la presencia de terceros. Quizás en un entorno de laboratorio controlado esta ventaja parezca menor, pero entrenar el sistema bajo esta lógica asegura que el modelo sea capaz de transferir su aprendizaje a situaciones del día a día, donde las condiciones distan mucho de ser ideales.

Otro ostaculo típico es la oclusión, cuando las manos se cruzan, se superponen o tapan en parte el rostro. Alphapose tiene integrado nativamente dos mecanismos que buscan

solucionar este problema. Por un lado, la Transformación Espacial Simétrica (SSTN) reajusta y centra la silueta del usuario incluso si la caja delimitadora inicial se desvió por culpa de la propia oclusión. Por otro, el algoritmo de Supresión de No Máximos Paramétrica (*p-Pose NMS*) realiza una labor de limpieza automática, eliminando extremidades duplicadas o falsos positivos que el sistema podría haber imaginado por confusión.

A todo esto se suma la capacidad del *software* para manejarse con vídeos borrosos o con cambios bruscos de iluminación. Cuando la nitidez decae debido a la rapidez de un gesto, entra en acción su módulo de rastreo temporal, *Pose Flow* (*Efficient Online Pose Tracking*). Este componente rompe con la rigidez de analizar imágenes aisladas y prefiere conectar los fotogramas vecinos para dar continuidad al esqueleto. Si un punto clave desaparece durante un par de cuadros por culpa del desenfoque, el sistema se apoya en el contexto inmediato anterior y posterior para recomponer la trayectoria, evitando deformaciones anómalas.

El desafío del *jittering* y la delegación temporal

Conviene aclarar que, pese a las virtudes de *Pose Flow* para mantener el hilo de las identidades, este módulo no realiza un pulido cinematográfico fino. El núcleo predictivo de AlphaPose sigue operando fotograma a fotograma, una inercia técnica que suele introducir micro-vibraciones (*jittering*) de unos pocos píxeles en las coordenadas resultantes.

En la visión por computador tradicional, la respuesta automática a este problema suele ser el uso de filtros matemáticos temporales (como el de Savitzky-Golay) o el diseño de redes auxiliares para forzar un alisado artificial de las trayectorias. Sin embargo, aplicar un borrón estadístico sobre la lengua de signos es peligroso, corremos el riesgo de maquillar o suprimir micro-movimientos que encierran un valor gramatical o semántico determinante (como la tensión en los dedos o un leve cambio de orientación).

Por este motivo, en este trabajo se ha tomado la decisión de no suavizar artificialmente los datos espaciales durante el preprocesamiento. Preferimos delegar esa incertidumbre en la propia arquitectura de la red predictiva (los flujos de *MSKA* y sus módulos de atención). Confiamos en que la capacidad de la arquitectura *Transformer* para capturar dependencias temporales largas mediante la autoatención sea suficiente para absorber este ruido, logrando dar sentido a las fluctuaciones sin sacrificar la riqueza expresiva del signo original.

Configuración técnica y formato de salida

Alphapose tiene la capacidad de cambiar la topología que representan los puntos clave del esqueleto. Existen principalmente dos opciones adecuadas para esta tarea: **COCO-WholeBody**, una extensión del clásico MS-COCO que cubre el cuerpo entero con 133 puntos, y **HALPE-136**, que proporciona una anotación de 136 puntos clave. Elegimos esta última opción porque introduce anclas de referencia espacial estratégicas en el torso, como el centro de la cadera, el cuello y la cabeza. Este mapa de puntos optimiza el seguimiento temporal en secuencias dinámicas y actúa como un contrapeso que estabiliza la

captura de gestos manuales veloces o marcadores faciales, mitigando en parte el problema del *jittering*.

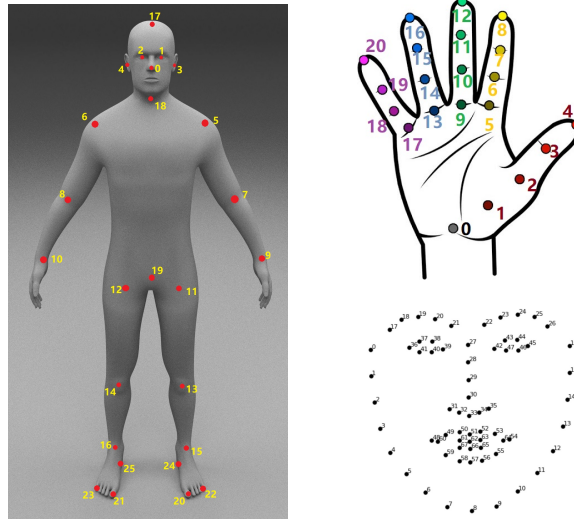


Figura 4.1: Distribución de *keypoints* anatómicos en el formato Halpe-136 [7].

El producto final tras procesar cada vídeo es un archivo estructurado en formato JSON. Cada fotograma se traduce en un diccionario que almacena la lista plana de coordenadas bidimensionales (x, y) combinadas con su respectivo índice de confianza (c) para cada punto anatómico detectado, reflejando la estructura del siguiente fragmento:

```
[
  // Primera persona, primera imagen
  {
    "image_id" : string, image_1_name,
    "category_id" : int, 1 for person
    "keypoints" : [x1,y1,c1,...,xk,yk,ck],
    "score" : float,
  },
  // Segunda persona, primera imagen
  {
    "image_id" : string, image_1_name,
    "category_id" : int, 1 for person
    "keypoints" : [x1,y1,c1,...,xk,yk,ck],
    "score" : float,
  },
  ...
  // Igual para la segunda imagen
  {
    "image_id" : string, image_2_name,
    "category_id" : int, 1 for person
    "keypoints" : [x1,y1,c1,...,xk,yk,ck],
    "score" : float,
  },
]
```

...
]

4.2. Normalización de los Datos

Las coordenadas brutas producidas por AlphaPose se expresan en píxeles absolutos del fotograma, lo que las hace dependientes de la resolución del vídeo de origen. Para que el modelo pueda aprender representaciones independientes de dicha resolución y para favorecer la estabilidad numérica durante el entrenamiento, se aplica una normalización espacial a todos los *keypoints* antes de introducirlos en la red.

El proceso consta de dos etapas. En primer lugar, las coordenadas se llevan al rango $[0, 1]$ dividiendo cada componente entre las dimensiones del fotograma: la coordenada x se divide entre el ancho W y la coordenada y entre el alto H .

Dado que en el sistema de coordenadas de imagen el eje y crece hacia abajo, se invierte previamente dicho eje,

$$y \leftarrow H - y$$

para alinearlos con la convención matemática estándar, en la que el origen se sitúa en la parte inferior.

En segundo lugar, el rango $[0, 1]$ se reescala al intervalo $[-1, 1]$ mediante la transformación

$$v \leftarrow \frac{v - 0,5}{0,5},$$

de modo que el centro geométrico del fotograma queda mapeado al origen de coordenadas. Esta elección es especialmente conveniente en el dominio de la lengua de signos, ya que el torso del signante, que actúa como referencia espacial global, tiende a ocupar la región central del encuadre.

El tercer canal de cada punto clave, correspondiente al índice de confianza c asignado por el estimador de pose, no se modifica, pues ya se encuentra acotado en el rango $[0, 1]$ por diseño del modelo de extracción.

4.3. *Multi-Stream Keypoint Attention*

En esta sección se presentará la arquitectura principal del modelo *Multi-stream Keypoint Attention* (MSKA) detallando el razonamiento detrás de la misma. Es importante notar que este modelo **no** es *end-to-end*, necesita la representación en glosas, se utilizan para asegurar que el modelo realmente aprende el significado detrás de los *keypoints*.

4.3.1. Data Augmentation

Como mencionábamos en la *ch:SOTA*, existe una falta de datos, los conjuntos de datos disponibles son escasos y apenas tienen, por ejemplo en el caso de Phoenix-2014T unos

7000 videos de entrenamiento. Así, y con la intención de reducir el sobreajuste se decide utilizar técnicas de aumento de datos clásicas en problemas que tratan con *keypoints*.

La postura y la ubicación del intérprete varían de forma natural al realizar un signo. Esto permite introducir variabilidad controlada en los datos sin corromper su semántica ni generar muestras inverosímiles. Entre las transformaciones espaciales más comunes y efectivas se encuentran las **rotaciones**, las **traslaciones** y los **cambios de escala (zoom)**.

Sorprendentemente, los autores observaron que la eficacia del modelo comenzó a disminuir específicamente al integrar los métodos de traslación y cambio de escala. Su hipótesis es que estas estrategias de aumento introdujeron discrepancias en la alineación de los datos con el conjunto de validación. En lugar de ayudar a generalizar, estas alteraciones terminaron provocando un sobreajuste. Así, decidieron mantener únicamente las rotaciones.

Además de estos métodos, proponemos dos métodos de aumento de datos y regularización específicos para la tarea de reconocimiento de la pose: **Joint Dropout** y **ruido gaussiano**, su funcionamiento y la motivación de su uso se explicará en las secciones siguientes.

Rotaciones

El objetivo es que el modelo sea robusto ante pequeñas variaciones en la inclinación o el ángulo del intérprete. Para lograrlo, se aplica una rotación aleatoria de α grados con una probabilidad p . Si una matriz $\mathbf{P} \in \mathbb{R}^{n \times 2}$ contiene las coordenadas (x, y) de los puntos clave de un fotograma, la transformación se calcula mediante el siguiente producto matricial.

$$\begin{bmatrix} x'_0 & y'_0 \\ x'_1 & y'_1 \\ \vdots & \vdots \\ x'_n & y'_n \end{bmatrix} = \begin{bmatrix} x_0 & y_0 \\ x_1 & y_1 \\ \vdots & \vdots \\ x_n & y_n \end{bmatrix} \cdot \underbrace{\begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}}_{\text{Matriz de rotaciones traspuesta}}$$

Nota 4.3.1. Al multiplicar la matriz de coordenadas (donde cada fila es un punto) por la derecha, necesitamos la matriz de rotación traspuesta. La traspuesta de la matriz de rotación estándar para un ángulo α en sentido antihorario cambia el signo del seno inferior al seno superior. Esto nos permite realizar la rotación del fotograma en un paso.

Es importante acotar el rango de esta transformación; rotaciones extremas (por ejemplo, de 180°) carecen de sentido práctico, ya que introducen escenarios que el modelo jamás encontrará en un entorno real. Por eso nos limitamos a una rotación de $\pm 15^\circ$.

Aumento Temporal

Las manos ocupan un área reducida del video y son sumamente vulnerables a los movimientos rápidos que ocurren de manera natural durante la comunicación en lenguaje de señas. La idea al alterar aleatoriamente la longitud de la secuencia, es que el modelo se

vea forzado a aprender las características de estos movimientos sin importar la velocidad a la que ocurran.

Para modificar la longitud temporal de las secuencias de puntos clave se seleccionan fotogramas válidos de forma completamente aleatoria dentro de un rango determinado. Si seleccionamos una proporción < 1 de fotogramas simulamos a la persona hacer la misma seña, pero a mayor de velocidad. Si la proporción es > 1 se está añadiendo redundancia o manteniendo la información de los fotogramas por más tiempo simulamos a la persona hacer la misma seña pero más lentamente.

A nivel de implementación, la expansión de la secuencia se realiza duplicando fotogramas de forma aleatoria, mientras que su reducción se logra descartándolos, preservando en todo momento el orden cronológico original.

Joint Dropout

Siguiendo la idea que proponen en *Leveraging Artificial Occluded Samples for Data Augmentation in Human Activity Recognition* [27], buscamos simular la pérdida de información que ocurre cuando una o más de las extremidades del sujeto están ocultas por algún obstáculo o el modelo de extracción de puntos clave es incapaz de extraer la información de algunas extremidades.

El método consiste en eliminar la información de ciertos keypoints agrupándolos por extremidad, por ejemplo *mano derecha*, *mano izquierda*, *brazo izquierdo* y *brazo derecho*. Además los investigadores señalan que la oclusión debe afectar la duración completa de la acción para forzar a la red a aprender de las extremidades restantes.

Se implementa una versión simplificada de la técnica, que trata con datos $2D$ a diferencia de los datos $3D$ con los que se trata en el papel mencionado. Como detalle, no tiene sentido la oclusión total de las extremidades, ya que esto privaría a la red de características representativas e introduciría ruido perjudicial en el entrenamiento. Así, basandonos en [27] nos limitamos a eliminar 2 extremidades como máximo.

Ruido Gaussiano

Otra situación que ocurre frecuentemente en este tipo de tareas son los micromovimientos de la persona, las interferencias del entorno y los temblores naturales que ocurren al capturar el movimiento. El objetivo principal de implementar el ruido gaussiano es simular estas perturbaciones comunes. Nos basamos en parte en *Enhancing Human Action Recognition with 3D Skeleton Data: A Comprehensive Study of Deep Learning and Data Augmentation* [28].

Se añade ruido que sigue una distribución normal $\mathcal{N}(\mu, \sigma^2)$. Para asegurar que el efecto del ruido sea perceptible pero sin llegar a distorsionar gravemente los datos originales, los autores configuraron la media, μ , en 0 y la desviación estándar σ en 0,01.

4.3.2. Arquitectura del Modelo de Reconocimiento

La lengua de signos no concentra el significado en una sola articulación. Sino que lo distribuye a través de distintos canales simultáneamente: las configuraciones de las manos, los movimientos de los brazos y las expresiones faciales. Para aprovechar esta riqueza de información, el sistema utiliza un mecanismo de autoatención que le permite analizar cada flujo de manera independiente. Posteriormente, combina la información obtenida de cada flujo mediante técnicas de destilación de conocimiento para construir una representación más completa y precisa.

La arquitectura de este módulo, cuyos componentes se detallan en los apartados subsiguientes, se ilustra en el siguiente diagrama.

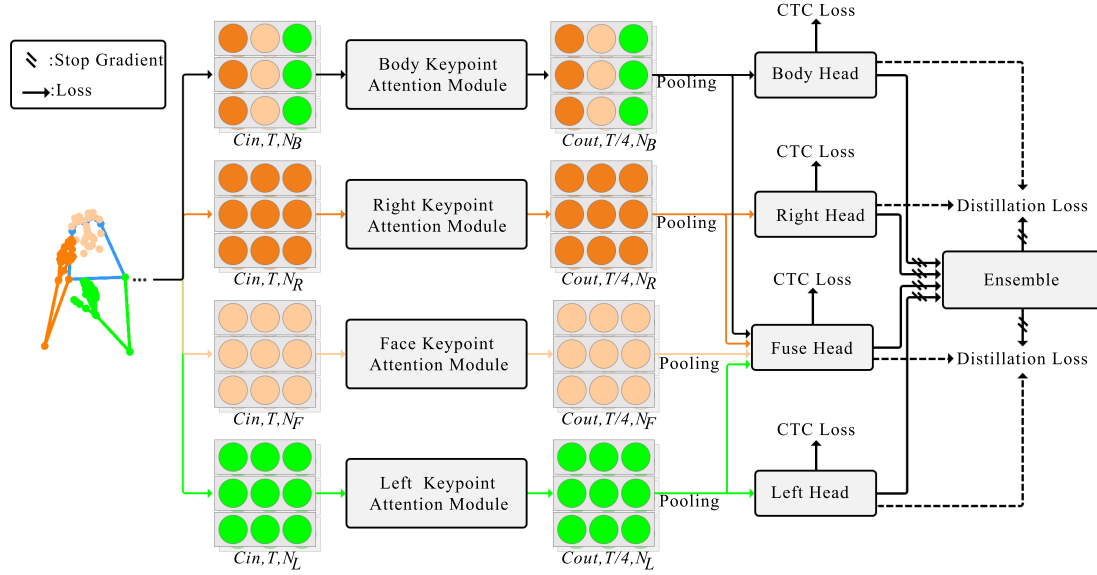


Figura 4.2: Esquema de la estructura del módulo de reconocimiento del modelo. Imagen de [19]

Desacoplamiento de Keypoints

Como mencionábamos el modelo **MSKA-SLR** deja de utilizar un único flujo de procesamiento (*stream*) y adopta una estrategia de separación de puntos clave inspirada en los mecanismos de percepción humana.

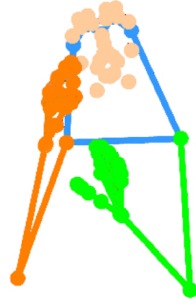


Figura 4.3: Esquema de la división de las extremidades. Imagen de [19]

Como se observa en la figura anterior, la secuencia global extraída del vídeo se segmenta en cuatro flujos (*streams*) independientes: **mano derecha**, **mano izquierda**, **rostro** y **cuerpo entero**.

Este modelado paralelo captura las características específicas de cada región anatómica, mitigando el sesgo de los enfoques monolíticos donde los movimientos amplios del cuerpo pueden eclipsar las variaciones sutiles de los dedos o de la boca, que contienen información muy relevante. La separación asegura que los detalles finos no se pierdan en la representación global.

Además, el procesamiento independiente permite que la red asigne implícitamente la importancia adecuada a cada fuente de información durante la etapa posterior de fusión.

Una vez divididos los flujos, cada subsecuencia se procesa mediante su propio módulo de atención de puntos clave (*Keypoint Attention Module*).

Módulo de Atención de Keypoints

Este módulo integra los mecanismos de atención espacio-temporal descritos en la Sección 3.4. Los pesos de estos bloques son independientes y no se comparten entre los distintos flujos. El siguiente diagrama ilustra la adaptación de esta estructura a los *keypoints* corporales.

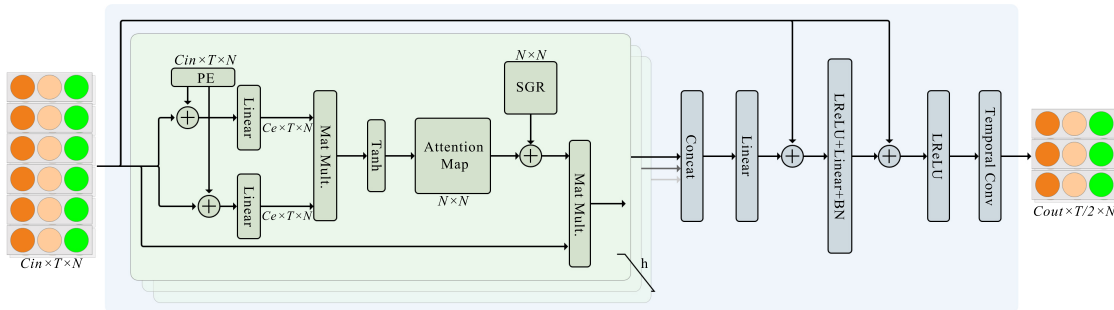


Figura 4.4: Esquema del módulo de atención de *keypoints* del cuerpo. Se toma $h = 6$. Imagen de [19]

El flujo de datos comienza con una secuencia de puntos clave estructurada como un tensor multidimensional $X \in \mathbb{R}^{C \times T \times N}$, donde:

- C representa los **canales de entrada**, definidos por el vector $[x_t^n, y_t^n, c_t^n]$. Las variables (x_t^n, y_t^n) denotan las coordenadas espaciales bidimensionales, mientras que c_t^n indica el nivel de confianza asignado por el estimador de pose al punto clave n en el instante t .
- T corresponde a la dimensión temporal (número total de fotogramas).
- N es el número total de puntos clave específicos del flujo.

A diferencia de la arquitectura *DSTA* descrita en la *Sección 3.4*, la codificación posicional se restringe a la dimensión espacial, delegando el modelado temporal a convoluciones $2D$ en etapas posteriores.

Esta modificación evita el incremento de parámetros inherente a la atención temporal. Dado el tamaño reducido de los conjuntos de datos de lenguaje de señas, una alta parametrización elevaría significativamente el riesgo de sobreajuste.

Delegado el modelado temporal a las convoluciones $2D$, el tensor se procesa mediante una cascada de ocho bloques de atención independientes por flujo. Cada bloque incorpora una convolución temporal al final de su estructura; no obstante, solo el primer y quinto bloque aplican un paso (*stride*) de 2. Esta configuración contrae la resolución temporal de forma escalonada, reduciendo los fotogramas de T a $T/4$ ($T \rightarrow T/2 \rightarrow T/4$). Simultáneamente, la dimensión de los canales de entrada (C_{in}) se expande progresivamente (64, 128 y 256) hasta fijar un valor de salida $C_{out} = 256$.

Finalizada la cascada de atención, el tensor resultante $\in \mathbb{R}^{256 \times T/4 \times N}$ es procesado mediante un agrupamiento espacial (*Spatial Pooling*) antes de alimentar a la red de cabecera. Esta operación promedia la dimensión correspondiente a los *keypoints* (N), colapsando la representación espacial explícita del esqueleto. Se obtiene así un tensor bidimensional de $T/4 \times 256$ que sintetiza el espacio en características abstractas, aislando la dinámica temporal para la clasificación final de las glosas.

Redes de Cabecera (*Head Networks*) y Fusión

Cada flujo de procesamiento (mano izquierda, mano derecha, rostro y cuerpo) tiene su propia red de cabecera independiente. Estas estructuras modelan el contexto temporal final de los datos, extrayendo la dinámica global del movimiento a lo largo del tiempo.

Cada cabeza está compuesta por una capa lineal temporal, una de normalización por lotes (*batch normalization*), una de activación *ReLU* y un bloque convolucional temporal, que contiene dos capas de convolución $1D$ con un tamaño de kernel de 3. Termina con otra lineal y un último *ReLU*.

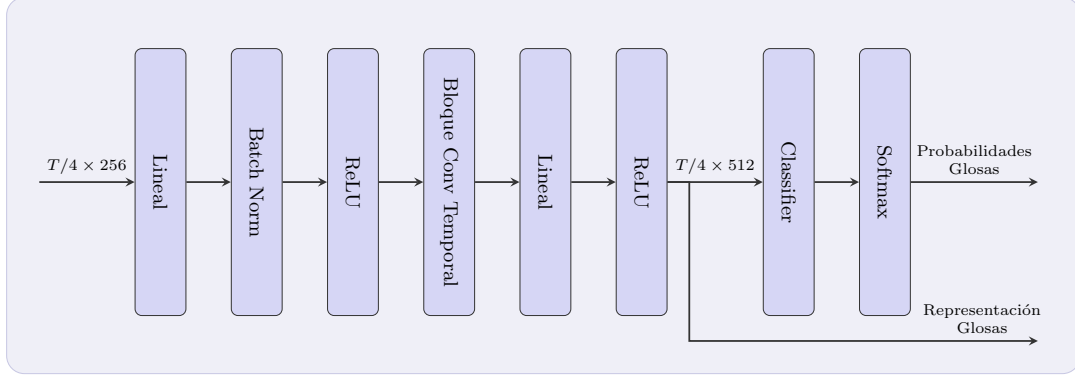


Figura 4.5: Esquema las redes de cabecera.

Produce lo que los autores denominan la **representación de la glosa**, con dimensiones $T/4 \times 512$. Finalmente, un clasificador lineal acoplado a una activación *softmax* estima la distribución de probabilidades para predecir la glosa articulada. Esta probabilidad se utiliza principalmente para el cálculo de la pérdida; el modelo de traducción recibe la representación de la glosa.

Cada cabezal se entrena y se corrige utilizando la función de pérdida CTC expuesta en la *Sección 3.3* de manera independiente.

Para aprovechar al máximo la separación de la información espacial, la arquitectura incorpora un cabezal auxiliar denominado **Cabezal de Fusión** (*Fuse Head*). Su objetivo es asimilar e integrar las características extraídas por los cuatro flujos independientes (manos, rostro y cuerpo) antes de emitir una decisión final. La estructura interna de este módulo de fusión es idéntica a la de las redes de cabecera individuales y, de igual manera, es supervisada por su propia función de pérdida CTC ($\mathcal{L}_{CTC}^{fuse}$).

Durante la inferencia, las probabilidades de glosa a nivel de fotograma predichas por los distintos flujos se promedian y se envían a un módulo de *ensemble* para decodificar la secuencia final. Este enfoque unifica las representaciones locales, mejorando significativamente la robustez y precisión de las predicciones.

Función de Pérdida y Autodestilación

El entrenamiento conjunto de la red *MSKA-SLR* está guiado por dos componentes principales. En primer lugar, se aplica la pérdida CTC a las salidas de cada una de las cuatro cabezas.

$$\mathcal{L}_{CTC} = \mathcal{L}_{CTC}^{left} + \mathcal{L}_{CTC}^{right} + \mathcal{L}_{CTC}^{body} + \mathcal{L}_{CTC}^{fuse}$$

En segundo lugar, para complementar la naturaleza global de la pérdida CTC y lograr una interacción más profunda entre los flujos, el modelo implementa una **autodestilación**

a nivel de fotograma. Esta técnica calcula el promedio de las probabilidades emitidas por las cuatro redes de cabecera principales y lo utiliza como un *pseudo-objetivo*.

A través de una pérdida de destilación ($\mathcal{L}_{Dist}$), se minimiza la divergencia KL (*Kullback-Leibler*)¹ entre este pseudo-objetivo unificado y las predicciones individuales de cada flujo.

Para asegurar la estabilidad del entrenamiento, se aplica una operación de **parada de gradiente** sobre el ensamblaje. Al desconectar el *pseudo-objetivo* del grafo computacional durante la retropropagación, se garantiza que la pérdida de destilación actúe únicamente sobre los pesos de los flujos individuales. Esto evita la retroalimentación del error hacia el ensamblaje, previniendo el colapso del modelo y asegurando que el objetivo de referencia actúe como un ancla estática.

Así, la función de pérdida global es la siguiente.

$$\mathcal{L}_{SLR} = \mathcal{L}_{CTC} + \mathcal{L}_{Dist}$$

De este modo, el conocimiento unificado del ensamblaje se transfiere a cada flujo individual. Esto garantiza que cada cabezal no solo aprenda de sus propias características locales, sino que optimice sus predicciones basándose en la comprensión global de toda la red.

4.3.3. Arquitectura del módulo de traducción

Los autores expanden el modelo de reconocimiento hacia un modelo de traducción mediante la adición de una red de traducción externa. La arquitectura sigue un paradigma de codificador-decodificador.

La red MSKA-SLR hace papel de codificador visual (*Visual Encoder*), su salida, la **representación de la glosa**, sirve como el “lenguaje intermedio” que entiende la red.

El responsable de generar la secuencia textual final es el decodificador (*Translation Network*). Exploraremos la arquitectura original y la expandiremos utilizando **modelos de lenguaje**.

Integración mediante MLP

Dado que el codificador visual opera en un dominio de características espaciales y temporales comprimidas ($T/4 \times 512$), es necesario adaptar esta salida antes de introducirla en el decodificador de lenguaje.

Para este fin, el sistema emplea un **perceptrón multicapa** (*MLP*) con dos capas ocultas. Este bloque actúa como un adaptador de dominio muy básico: proyecta las representaciones de la glosa al espacio semántico que el modelo de lenguaje espera, alineando las características extraídas por el flujo de fusión con las entradas del decodificador.

¹La divergencia de Kullback-Leibler (KL), también llamada entropía relativa, es una medida estadística que cuantifica cuánto difiere una distribución de probabilidad de otra [29].

MBART como decodificador

El artículo opta por utilizar **mBART** (*Multilingual BART*) [30] como la red de traducción principal.

mBART es un modelo de lenguaje preentrenado con una arquitectura tipo *transformer encoder-decoder* que ha demostrado un rendimiento sobresaliente en tareas de traducción automática multilingüe.

Gracias a su preentrenamiento sobre grandes corpus multilingües, el modelo ya codifica las estructuras sintácticas y semánticas necesarias para la decodificación a lenguaje natural, lo que reduce el entrenamiento a alinear las representaciones visuales con su espacio lingüístico.

La pérdida de traducción se calcula mediante entropía cruzada a nivel de token (*token-level cross-entropy loss*): el decodificador predice la secuencia objetivo de forma autorregresiva, optimizando la probabilidad de cada token correcto dado el contexto previo y la representación visual codificada.

Modelo de lenguaje pequeño como decodificador

Los modelos *decoder-only* modernos han demostrado capacidades de generalización, traducción y comprensión lingüística superiores a muchos modelos *encoder-decoder*, gracias a su preentrenamiento sobre corpus de texto de mayor escala. Así, en lugar de un modelo *encoder-decoder* como mBART, optamos por integrar un modelo de lenguaje con arquitectura exclusivamente de decodificador (*decoder-only*).

En esta arquitectura, las características visuales no pasan por un codificador independiente. En su lugar, se emplea una estrategia de **prefijo visual** (*visual prefix conditioning*), inspirada en modelos multimodales como LLaVA [35]:

1. El *VLMapper* proyecta las características visuales al espacio de embeddings del modelo de lenguaje.
2. Estas características se tratan como una secuencia de tokens continuos y se concatenan al inicio de la entrada.
3. Se anexa un *prompt* textual de instrucción que guía al modelo hacia la tarea de traducción.

La secuencia unificada que procesa el modelo adopta la estructura:

[Tokens Visuales] + [Prompt Textual] + [Traducción Objetivo].

El módulo de proyección visual (*VLMapper*)

El *VLMapper* es el puente entre el espacio de representaciones del reconocedor visual y el espacio de *embeddings* del modelo de lenguaje. Su función es proyectar la secuencia de características producida por MSKA-SLR, de dimensión $T/4 \times 512$, al espacio semántico

en que opera Qwen, cuya dimensión de *embedding* es d_{LM} (1536 para Qwen2.5-1.5B, 3584 para Qwen2.5-7B).

Arquitectónicamente, el *VLMapper* es un **perceptrón multicapa (MLP)** de dos capas con activación GELU entre ellas:

$$\text{VLMapper}(\mathbf{H}) = W_2 \cdot \text{GELU}(W_1 \mathbf{H} + b_1) + b_2$$

donde $\mathbf{H} \in \mathbb{R}^{(T/4) \times 512}$ es la representación de la glosa producida por MSKA-SLR, $W_1 \in \mathbb{R}^{512 \times 512}$, $W_2 \in \mathbb{R}^{512 \times d_{\text{LM}}}$.

La salida del *VLMapper* es, por tanto, una secuencia de $T/4$ vectores en $\mathbb{R}^{d_{\text{LM}}}$, que se tratan como **tokens visuales continuos** y se concatenan al inicio del contexto del modelo de lenguaje.

El *VLMapper* se entrena conjuntamente con los adaptadores LoRA durante el *finetuning*: sus pesos **no se congelan**, de modo que puede ajustarse para alinear progresivamente el espacio visual con el espacio lingüístico de Qwen a lo largo del entrenamiento. Esta decisión de diseño está motivada por el bajo número de parámetros del proyector ($\approx 0,5$ M para Qwen2.5-1.5B y $\approx 1,8$ M para Qwen2.5-7B), que hacen viable su entrenamiento completo sin riesgo de sobreajuste.

Modelo LM	d_{LM}	Parámetros del VLMapper (M)
Qwen2.5-1.5B	1536	$\approx 0,5$
Qwen2.5-7B	3584	$\approx 1,8$

Tabla 4.1: Dimensión de *embedding* del modelo de lenguaje y parámetros del *VLMapper* para cada variante experimental.

Entrenamiento

La función de pérdida se aplica únicamente sobre los tokens de la traducción objetivo. De este modo, el modelo no recibe penalización por su comportamiento sobre el contexto de entrada, concentrando toda la señal de supervisión en aprender a generar la traducción correcta condicionada a la información visual.

Dado que un *finetuning* completo de un modelo de lenguaje podría desestabilizaría sus capacidades lingüísticas preentrenadas y exigiría recursos computacionales prohibitivos, la red de traducción se adapta mediante **LoRA** (explicado en Sección 3.5).

Antes de explorar dicha estrategia, evaluaremos su capacidad en un escenario *zero-shot*, siguiendo la línea del trabajo **Spotter+GPT** [20] presentado en el estado del arte. En dicho trabajo, las glosas reconocidas se pasan directamente a un LLM mediante un *prompt* de instrucción, sin ningún tipo de entrenamiento adicional, obteniendo traducciones coherentes gracias a la capacidad lingüística preentrenada del modelo.

4.3.4. Tokenizador

Todo sistema de traducción automática requiere convertir las secuencias de texto en representaciones numéricas que la red pueda procesar. En este trabajo coexisten dos niveles de tokenización con propósitos distintos: uno para las glosas, que actúa como señal de supervisión del módulo de reconocimiento, y otro para el texto en lenguaje natural, que supervisa el módulo de traducción.

Tokenizador de Glosas

Como mencionamos, las glosas son la representación intermedia del lenguaje de signos, secuencias de palabras clave normalmente en mayúsculas que describen los signos ejecutados.

Su tokenización se realiza mediante un vocabulario cerrado construido a partir del conjunto de entrenamiento, donde cada glosa única recibe un identificador numérico. Este vocabulario se emplea tanto para convertir las etiquetas de referencia en índices durante el entrenamiento como para decodificar las predicciones del módulo CTC durante la inferencia.

Tokenizador de mBART

En la variante con mBART como red de traducción, el texto en alemán se tokeniza con el tokenizador propio del modelo, basado en *SentencePiece*. Al tratarse de un modelo *encoder-decoder* multilingüe, requiere un tratamiento específico: se añade un token de idioma de destino al inicio de la secuencia del decodificador (`de_DE`), y las posiciones de *padding* se marcan con `-100` para que sean ignoradas durante el cálculo de la entropía cruzada. Además, las secuencias de entrada al decodificador se desplazan una posición a la derecha (*shift_tokens_right*), siguiendo el paradigma de *teacher forcing* propio de los modelos autorregresivos *encoder-decoder*.

Tokenizador de Qwen

En la variante con un modelo de lenguaje *decoder-only*, se emplea el tokenizador de **Qwen**. A diferencia de mBART, este tokenizador no requiere token de idioma ni desplazamiento de la secuencia, el propio modelo gestiona internamente el *teacher forcing* a partir de los *labels*.

La secuencia que se tokeniza concatena un *prompt* de instrucción seguido del texto objetivo, siguiendo la estructura descrita en la sección anterior. La pérdida se enmascara sobre los tokens del *prompt* asignándoles el valor `-100`, de modo que la señal de supervisión recaer exclusivamente sobre los tokens de la traducción.

Capítulo 5

Experimentación y análisis de resultados

Este capítulo presenta los experimentos realizados para validar las decisiones de diseño descritas en la metodología y evaluar el rendimiento del sistema propuesto sobre el conjunto de datos PHOENIX-2014T.

La experimentación se organiza en tres bloques progresivos. En primer lugar, se estudia el módulo de reconocimiento de forma aislada, analizando el impacto de las distintas técnicas de aumentación de datos sobre la tasa de error en glosas. En segundo lugar, se evalúa la capacidad de traducción de Qwen en un escenario *zero-shot*, estableciendo una línea base comparable al enfoque Spotter+GPT descrito en el estado del arte. Finalmente, se presenta el sistema completo con *finetuning* mediante LoRA, incluyendo un estudio de ablación sobre las modalidades de entrada y una comparativa con la variante basada en mBART.

Para la métrica BLEU se utiliza la implementación propuesta en *A Call for Clarity in Reporting BLEU Scores* [21].

5.1. Configuración experimental

5.1.1. Entorno de entrenamiento

Todos los experimentos se ejecutaron sobre una única **GPU NVIDIA A100** de 40 GB de memoria, con precisión mixta `bf16` para reducir el consumo de memoria sin sacrificar estabilidad numérica durante el entrenamiento.

El código está desarrollado en Python con PyTorch como *framework* principal, y se emplea la librería *Transformers* de Hugging Face para la gestión de los modelos de lenguaje y sus tokenizadores. Todo el código está disponible en el repositorio de GitHub: <https://github.com/nicolasrodriguezruiz/tfm-sign-language>

5.1.2. Hiperparámetros generales

Los hiperparámetros comunes a todos los experimentos se recogen en la Tabla 5.1.

Parámetro	Reconocimiento (S2G)	Traducción (S2T)
Tamaño de batch	16	8
Épocas máximas	100	40
<i>Early stopping</i>	15 épocas	10 épocas
Optimizador	AdamW	
β_1, β_2	0.9, 0.998	
<i>Weight decay</i>	0.02	
<i>Scheduler</i>	Annealed Cosine Decay	

Tabla 5.1: Hiperparámetros comunes a todos los experimentos.

Los *learning rates* específicos de cada variante se detallan en la sección correspondiente, dado que varían según el módulo entrenado y la estrategia de ajuste empleada.

5.1.3. Configuración de LoRA

Para el *finetuning* de los modelos Qwen se emplea LoRA, manteniendo los pesos base del modelo congelados e inyectando matrices de bajo rango únicamente en los módulos de atención.

- **LoRA-Atención:** adaptadores únicamente en las proyecciones de atención: `q_proj`, `k_proj`, `v_proj`, `o_proj`.
- **LoRA-Completo:** adaptadores adicionales en las capas del bloque *feed-forward*: `gate_proj`, `up_proj`, `down_proj`.

Los parámetros de LoRA son comunes a ambas configuraciones y se recogen en la Tabla 5.2.

Parámetro	Valor
Rango (r)	64
Alpha (α)	128
Dropout	0.25

Tabla 5.2: Configuración de LoRA empleada en los experimentos de traducción.

La escala efectiva de los adaptadores queda determinada por el cociente $\alpha/r = 2$, lo que implica que las actualizaciones de bajo rango se aplican con un factor de escala de 2 sobre los pesos originales.

En el caso de **Qwen2.5-1.5B**, el tamaño reducido del modelo permite explorar tanto el *finetuning completo* como el ajuste mediante **LoRA**. El *finetuning* completo sirve

como referencia para cuantificar el coste real de capacidad que supone la aproximación de bajo rango.

Para los modelos de mayor tamaño (**Qwen2.5-7B** y **Qwen2.5-14B**), el *finetuning* completo resulta inviable bajo las restricciones de memoria de una única A100 de 40 GB, por lo que se emplea exclusivamente LoRA.

5.1.4. Configuración de generación

La generación de texto durante la inferencia se controla mediante los parámetros recogidos en la Tabla 5.3. Como es convencional, existen dos fases validación, empleada durante el entrenamiento para seleccionar el mejor *checkpoint*, y test, utilizada para la evaluación final sobre el conjunto de prueba.

Parámetro	Validación	Test
<i>Reconocimiento (CTC beam search)</i>		
<i>Beam size</i>	1	5
<i>Traducción (Qwen)</i>		
<i>Beam size</i>	4	4
<code>max_new_tokens</code>	51	51
<code>length_penalty</code>	0.0	0.0
<code>repetition_penalty</code>	1.5	1.5
<code>no_repeat_ngram_size</code>	3	3
<code>temperature</code>	0.05	0.05
<code>do_sample</code>	false	false
<code>early_stopping</code>	true	true

Tabla 5.3: Parámetros de generación empleados en validación y test.

Durante la validación se usa *beam size* 1 para el reconocimiento con el objetivo de reducir el coste computacional por época, reservando la búsqueda en haz completa ($b = 5$) para la evaluación final.

En la traducción, `do_sample=false` fuerza una decodificación casi determinista, equivalente en la práctica a *beam search* puro. La penalización de repetición (1,5) y la restricción de n -gramas repetidos ($n = 3$) evitan el colapso de la generación hacia secuencias degeneradas, un problema especialmente frecuente cuando la entrada visual contiene ambigüedades o el reconocimiento introduce glosas incorrectas.

5.2. Análisis exploratorio del dataset

Antes de presentar los resultados de los modelos, es conveniente caracterizar cuantitativamente el conjunto de datos para justificar algunas de las decisiones de diseño adoptadas en la metodología.

5.2.1. Estadísticas del conjunto de entrenamiento

El conjunto de entrenamiento de PHOENIX-2014T contiene 7,096 muestras. La Tabla 5.4 resume las principales estadísticas sobre la longitud de los vídeos y de las frases en alemán, medidas estas últimas en tokens del vocabulario de Qwen2.5.

Estadístico	Frames por vídeo	Tokens por frase
Media	116.5	23.6
Desv. típica	49.8	9.7
Mínimo	16	3
Mediana (p50)	112	22
Percentil 75	144	29
Percentil 90	182	37
Percentil 95	208	41
Percentil 99	259	51
Máximo	475	78

Tabla 5.4: Estadísticas de longitud sobre el conjunto de entrenamiento de PHOENIX-2014T. Tokens medidos con el tokenizador de Qwen2.5.

Se observa que el percentil 99 de longitud textual se sitúa en 51 tokens, lo que justifica el valor `max_new_tokens=51` empleado durante la generación y `text_max_length=128` durante el entrenamiento, dejando margen suficiente para el prompt de instrucción.

En segundo lugar, el ratio medio de aproximadamente 5 frames por token implica que secuencias de hasta 400 frames corresponden a traducciones de en torno a 80 tokens, coherente con el límite `clip_len=400` establecido en el dataset.

5.2.2. Vocabulario de glosas

El vocabulario de glosas del conjunto de entrenamiento contiene 1094 entradas únicas, incluyendo los tokens especiales requeridos por la decodificación CTC. Cada glosa recibe un identificador numérico único construido a partir del conjunto de entrenamiento; las glosas no vistas durante el entrenamiento se mapean al token `<UNK>` en inferencia.

5.3. Reconocimiento de Lengua de Signos (SLR)

En esta sección se evalúa el módulo de reconocimiento de forma aislada sobre la tarea S2G, midiendo el impacto de las distintas técnicas de aumento de datos sobre la calidad del reconocimiento de glosas.

Métricas de evaluación

El rendimiento del reconocimiento se mide mediante la **Tasa de Error de Palabras** (*Word Error Rate*, WER), explicada en la Sección 3.1.1.

Adicionalmente se reportan las tasas individuales de sustitución (*SUB*), eliminación (*DEL*) e inserción (*INS*) para caracterizar el tipo de error dominante.

Antes del cálculo, tanto las hipótesis como las referencias se normalizan mediante la función `clean_phoenix_2014_trans`, que estandariza mayúsculas y elimina artefactos de puntuación propios del formato de anotación de PHOENIX-2014T.

5.3.1. Modelo base

El modelo de reconocimiento sigue la arquitectura MSKA-SLR descrita en el Capítulo 4, entrenado desde pesos aleatorios exclusivamente sobre los datos de PHOENIX-2014T.

La configuración base no aplica ninguna técnica de aumentación de datos más allá de la aumentación temporal y la rotación aleatoria, presentes en todos los experimentos.

Los parámetros específicos del módulo de reconocimiento se recogen en la Tabla 5.5.

Parámetro	Valor
<i>Learning rate</i>	1×10^{-3}
<i>Scheduler</i>	Cosine Annealing
$T_{\text{máx}}$	100
<i>Beam size</i> (validación)	1
<i>Beam size</i> (test)	5
Método de <i>ensemble</i>	Uniforme
<i>Early stopping</i> (δ)	0.1

Tabla 5.5: Configuración del módulo de reconocimiento.

La arquitectura DSTA-Net procesa cuatro flujos independientes ya mencionados en Sección 4, cuyos índices de *keypoints* se detallan en la Tabla 5.6. La decisión final se obtiene promediando uniformemente las distribuciones de probabilidad de los cuatro flujos junto con la cabeza de fusión (*ensemble* uniforme).

Flujo	Keypoints	Dimensión de entrada
Mano izquierda	27	512
Mano derecha	27	512
Cuerpo	78	256
Cara	26	—
Fusión	136 (todos)	1024

Tabla 5.6: Configuración de los flujos de la arquitectura MSKA-SLR. La cara se integra como contexto en los flujos de las manos y la cabeza de fusión, sin cabeza de clasificación propia.

5.3.2. Estudio de aumentación de datos

Se evalúan cinco configuraciones de aumentación, manteniendo fijos el resto de hiperparámetros. Las tres técnicas estudiadas: *Joint Dropout*, ruido gaussiano y *Structured Dropout* se describen en detalle en la Sección 4.3.1.

La diferencia entre *Joint Dropout* y *Structured Dropout* es que mientras el primero enmascara articulaciones individuales de forma aleatoria e independiente, cada keypoint tiene una probabilidad fija de ser anulado, sin ninguna restricción anatómica. El segundo, agrupa los keypoints por extremidades completas tal y como se describe en la sección correspondiente.

Mientras *Structured Dropout* modela situaciones más realistas, los resultados a continuación muestran que el *Joint Dropout* obtiene un WER ligeramente inferior (27,24 % frente a 27,59 % en test).

La razón por la que ocurre esto es que enmascarar una extremidad completa durante toda la secuencia priva al modelo de demasiada información simultáneamente, esto sumado a la falta de datos hacen que el modelo tenga mayores dificultades al aprender.

Por esta razón decidí realizar una pequeña modificación, en vez de ocultar por completo la articulación, ocultar puntos aleatorios preservando siempre una fracción suficiente de la estructura de cada extremidad, ofreciendo una regularización más suave y efectiva en este escenario de datos limitados.

Configuración	Dev WER↓	Test			
		WER↓	SUB	DEL	INS
SLR-base	27.81	27.33	11.81	13.29	2.23
SLR- <i>JointDropout</i>	27.68	27.24	12.28	12.51	2.44
SLR- <i>StructuredDropout</i>	28.24	27.59	12.82	12.21	2.56
SLR- <i>GaussianNoise</i>	28.56	28.13	12.82	13.15	2.16

Tabla 5.7: Comparativa de técnicas de aumentación sobre PHOENIX-2014T. Mejor resultado en negrita.

Análisis de resultados

Los resultados de la Tabla 5.7 son poco prometedores, las técnicas de aumento de datos no aportan mejoras sustanciales en este conjunto de datos y, en varios casos, perjudican el rendimiento.

La única variante que mejora ligeramente sobre la base es **SLR-*JointDropout***, que reduce el WER en test de 27,81 % a 27,68 %, una mejora marginal de 0,13 puntos (en validación). Su efecto más notable no es la reducción del WER global, sino el reequilibrio entre los tipos de error, las eliminaciones caen de 13,29 % a 12,51 % a costa de un leve incremento en sustituciones (de 11,81 % a 12,28 %).

Esto sugiere que enmascarar puntos clave individualmente obliga al modelo a ser más atrevido, reduciendo la tendencia a omitir glosas cuando parte de la información espacial

no está disponible.

El **SLR-StructuredDropout**, obtiene un resultado ligeramente inferior al *JointDropout* (28,24 % vs 27,68 % en validación). No obstante, presenta el valor de eliminaciones más bajo de todos los experimentos (12,21 %), lo que indica que la oclusión estructurada por extremidades ayuda al modelo a mantener la secuencia de glosas completa, aunque introduce mayor confusión entre signos similares (mayor tasa de sustitución).

El **SLR-GaussianNoise** empeora el rendimiento base en casi un punto (28,56 % vs 27,81 %). Probablemente, en el corpus de PHOENIX, las grabaciones en estudio de calidad homogénea hacen que el ruido gaussiano introduzca una variabilidad artificial que no corresponde a ningún patrón real del conjunto de validación, perjudicando la generalización.

En conjunto, la ganancia máxima obtenida mediante aumentación es inferior a 0,1 puntos de WER. Además, observando las curvas de las pérdidas en entrenamiento y validación (*Figura 5.1*) deducimos que el modelo sufre de sobreajuste y que las técnicas de aumentación exploradas no lo logran mitigar de forma significativa.

Las curvas de entrenamiento descienden de forma pronunciada hasta valores cercanos a 15, mientras que las curvas de validación se estabilizan en torno a 45 desde la época 20, sin mejorar demasiado hasta que la parada temprana finalmente detiene el entrenamiento de forma anticipada.

Esta brecha persistente entre entrenamiento y validación es evidencia del sobreajuste que se mantiene en todas las configuraciones de aumentación probadas.

La gráfica de WER confirma la misma lectura, todas las variantes convergen hacia el mismo rango (27-29 %) y sus trayectorias son prácticamente indistinguibles a partir de la época 35. El *StructuredDropout* presenta una convergencia más lenta durante las primeras épocas, posiblemente porque enmascarar extremidades completas dificulta el aprendizaje inicial, aunque termina alcanzando un resultado comparable al resto.

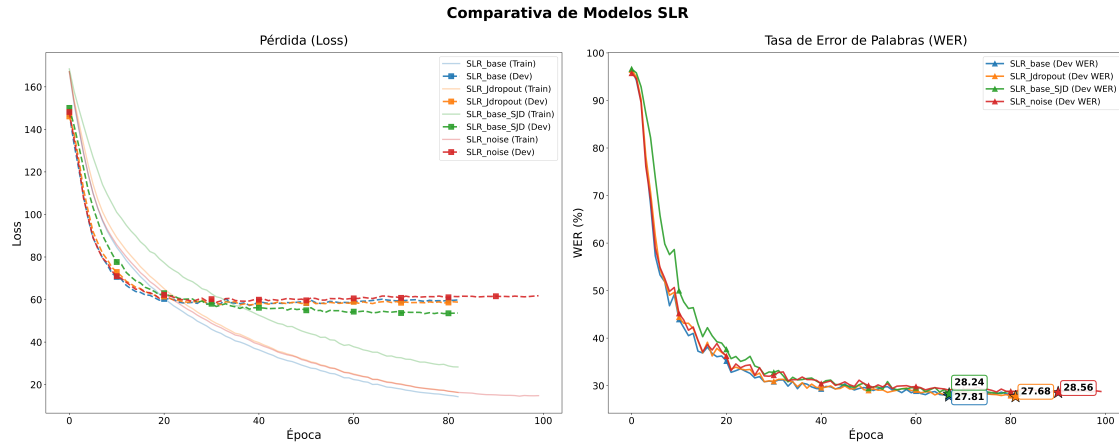


Figura 5.1: Evolución de la pérdida (izquierda) y del WER en validación (derecha) a lo largo del entrenamiento para las distintas configuraciones de aumentación. Las líneas transparentes en la gráfica de pérdida corresponden a las curvas de entrenamiento. Las estrellas marcan el *checkpoint* con el mejor modelo.

El mejor modelo, **SLR-JointDropout**, será el utilizado como módulo de reconocimiento en todos los experimentos de traducción posteriores.

Análisis cualitativo de las predicciones

Para complementar el análisis cuantitativo, la Tabla 5.8 recoge cuatro ejemplos representativos extraídos del conjunto de test, seleccionados para ilustrar los patrones de error más frecuentes observados en el mejor modelo (*JointDropout*).

Caso	Referencia	Predicción
Correcto	WAHRSCHEINLICH SCHAUER GEWITTER STARK	WAHRSCHEINLICH SCHAUER GEWITTER STARK
Sustitución	DEUTSCH LAND MORGEN HOCH DRUCK KOMMEN WOLKE AUFLOESEN	DEUTSCH LAND MORGEN HOCH DRUCK KOMMEN WOLKE VERSCHWINDEN
Eliminación	MORGEN LANG BLEIBEN	MORGEN
Fallo siste- mático	BIS ABEND IN-KOMMEND DANN KOMMEN NIEDE- RUNG	BIS NACHT IN-KOMMEND DANN KOMMEN

Tabla 5.8: Ejemplos cualitativos de predicciones del modelo SLR-*JointDropout* sobre el conjunto de prueba. En negrita las glosas donde se produce el error.

El primer caso confirma que el modelo funciona correctamente en secuencias cortas con

signos frecuentes en el corpus, donde la información de los keypoints es suficientemente discriminativa.

El error de sustitución del segundo ejemplo: *AUFLOESEN* (dispersarse) y *VERSCHWINDEN* (desaparecer) son sinónimos en el contexto meteorológico y comparten una configuración manual similar. Este tipo de confusión semántica entre glosas de significado próximo es difícilmente evitable desde el módulo de reconocimiento, y constituye precisamente uno de los casos donde un modelo de lenguaje con capacidad de razonamiento contextual puede recuperar el significado correcto en la etapa de traducción.

El tercer ejemplo ilustra el error de eliminación: el modelo reconoce la glosa inicial pero omite las dos siguientes, posiblemente porque la secuencia *LANG BLEIBEN* ocurre con menor frecuencia en el corpus y sus keypoints se confunden con ruido temporal.

El cuarto caso es el más significativo desde el punto de vista del análisis, todos los modelos evaluados, producen exactamente la misma predicción errónea sobre este ejemplo (y otros 123 del conjunto de prueba). La sustitución de *ABEND* por *NACHT* (tarde vs. noche) y la omisión de *NIEDERUNG* apuntan a que el error nace de una ambigüedad intrínseca en los datos: los signos correspondientes son visualmente similares y el corpus no proporciona suficientes ejemplos contrastivos para que el modelo aprenda a distinguirlos.

5.4. Traducción mediante mBART

El primer sistema de traducción evaluado es la variante original de MSKA acoplada a mBART, que actúa como línea base clásica y punto de referencia para todos los experimentos posteriores con modelos de lenguaje.

A diferencia de los sistemas basados en Qwen, mBART opera como un modelo *encoder-decoder* multilingüe especializado en traducción automática, por lo que no requiere prompt de instrucción ni estrategia de prefijo visual: las representaciones del codificador visual.

5.4.1. Resultados y análisis

Modelo	Dev WER↓	Dev					Test				
		B1	B2	B3	B4	R	B1	B2	B3	B4	R
MSKA-mBART	27.57	45.49	34.03	26.58	21.43	45.20	45.03	34.51	27.43	22.48	45.00

Tabla 5.9: Resultados de MSKA-mBART sobre PHOENIX-2014T. B1–B4: BLEU-1 a BLEU-4. R: ROUGE-L.

mBART establece una línea base sólida, alcanzando un BLEU-4 de 22,48 y un ROUGE-L de 45,00 en test. Estos valores son coherentes con los reportados en la literatura para sistemas basados en keypoints sobre PHOENIX-2014T, lo que confirma que la reimplementación del módulo de reconocimiento y el pipeline de preprocesado son correctos.

Destaca la consistencia entre los resultados de validación y test: la diferencia en BLEU-4 es inferior a un punto (21,43 vs 22,48), lo que indica que, a pesar de todo, el modelo

generaliza bien y no presenta sobreajuste en la etapa de traducción. Esta estabilidad contrasta con el comportamiento del módulo de reconocimiento, donde la brecha entre entrenamiento y validación era pronunciada, y sugiere que mBART aprovecha eficazmente su preentrenamiento multilingüe para compensar parcialmente los errores introducidos por el reconocedor.

Estos resultados servirán como referencia cuantitativa para evaluar si la sustitución de mBART por un modelo de lenguaje de arquitectura *decoder-only* supone una mejora real o implica un coste en calidad de traducción.

5.5. Traducción *zero-shot*: Spotter + Qwen

Antes de explorar el *finetuning*, se evalúa la capacidad de traducción de Qwen en un escenario *zero-shot*, siguiendo la línea del trabajo Spotter+GPT descrito en el estado del arte. En este experimento no se realiza ningún entrenamiento adicional: el módulo de reconocimiento predice una secuencia de glosas mediante CTC *beam search*, y dicha secuencia se pasa directamente a Qwen como texto plano mediante un prompt de instrucción, sin ningún tipo de entrenamiento ni acceso a features visuales.

5.5.1. Configuración

Se emplea **Qwen2.5-14B-Instruct** como modelo de lenguaje, elegido por su tamaño suficiente para exhibir capacidades de razonamiento *zero-shot* y por su disponibilidad bajo las restricciones de memoria de una única A100 de 40 GB en inferencia con precisión `bf16`.

El prompt de instrucción, formulado en alemán para maximizar la coherencia lingüística con la tarea basandonos en el de *Spotter+GPT* [20], tiene la siguiente estructura:

Du bist ein hilfreicher Assistent. Deine Aufgabe ist es, aus einer Liste von Wörtern einen sinnvollen deutschen Satz zu bilden. Beachte strikt diese Regeln:

1. Der Benutzer gibt dir nur eine Liste von Wörtern (Glossen). Bilde daraus einen fließenden Satz.
2. Antworte NUR mit dem generierten Satz. Wenn du keinen Satz bilden kannst, antworte mit 'Keine Übersetzung'.

Glossen:

Esto, traducido al español es:

Eres un asistente servicial. Tu tarea consiste en formar una oración alemana con sentido a partir de una lista de palabras. Sigue estrictamente estas reglas:

1. El usuario te proporciona únicamente una lista de palabras (glosas). Forma una oración fluida a partir de ellas.

2. Responda ÚNICAMENTE con la frase generada. Si no puede formar una frase, responda con “Sin traducción”.

Glosas:

Las glosas predichas por el módulo de reconocimiento se concatenan al final del prompt y se pasan al modelo mediante el formato nativo de chat (*ChatML*) de Qwen, utilizando `apply_chat_template` para garantizar la correcta inserción de los tokens de control.

5.5.2. Resultados y análisis

Modelo	B1	B2	B3	B4	R
Spotter+Qwen2.5-14B (<i>zero-shot</i>)	12.11	4.12	1.38	0.46	9.85

Tabla 5.10: Resultados del experimento *zero-shot* sobre el conjunto de test de PHOENIX-2014T. B1–B4: BLEU-1 a BLEU-4. R: ROUGE-L.

Los resultados cuantitativos sitúan el sistema en un BLEU-4 de 0,46 y un ROUGE-L de 9,85, valores muy inferiores a la línea base de mBART (22,48 y 45,00 respectivamente). Lo cual es razonable dado que el modelo en ningún momento se ha entrenado para predecir exactamente las frases del corpus. Con la métrica BLEURT, que mide similitud semántica en lugar de coincidencia exacta de n -gramas, el sistema alcanza un BLEURT de 0,5014, lo que indica que una fracción de las traducciones captura el significado correcto aunque con una formulación léxica distinta a la referencia. Como punto de comparación, el experimento con glosas de referencia (*ground truth*) en lugar de predichas obtiene un BLEURT de 0,5339, apenas 0,032 puntos por encima, lo que sugiere que el cuello de botella no reside únicamente en los errores del reconocedor sino principalmente en las limitaciones del modelo en capacidad *zero-shot*.

La Tabla 5.11 recoge ejemplos representativos que ilustran cualitativamente el comportamiento del sistema en ambos modos.

Modo	BLEURT	Glosas	Referencia	Predicción
Pred.	0.876 (mejor)	OST REGION MEISTENS SONNE	richtung osten ist es meist sonnig	In der Ostregion wird es meistens sonnig sein.
Pred.	0.075	PRED BLEIBEN	das bleibt auch noch in dieser woche so [...]	<i>Keine Übersetzung</i>
Pred.	0.071	IN-KOMMEND WIEDER KOM- MEN	die luft die in den kommenden tagen [...]	Inkommend wieder kommen. [el modelo añade una nota explicando que el contexto es insufi- ciente, incumpliendo la instrucción]
Pred.	0.049 (peor)	JETZT WIE- AUSSEHEN WET- TER MORGEN MITTWOCH DREI- ZEHN JANUAR	und nun die wetter- vorhersage für mor- gen mittwoch [...]	Morgen Mittwoch, den Dreizehn. Januar [...] <i>[alucinación en chino]</i>
GT	0.921 (mejor)	SAMSTAG SCH- NEE FALLEN	samstag fällt auch noch schnee	Am Samstag fällt noch Schnee.
GT	0.092	HEUTE NACHT ZWOELF REGION KOELN EINS REGION BRAN- DENBURG	heute nacht zwölf grad in der köln- bucht [...]	In der Nacht um zwölf Uhr in der Re- gion um Köln, eins und neun. [el modelo razona en voz alta en alemán, incumplien- do la instrucción]
GT	0.039 (peor)	DEUTSCHLAND IRLAND TIEF MITTE-MEER HOCH	deutschland liegt zwischen einem tief bei irland [...]	<i>[razonamiento en chino antes de responder Keine Übersetzung]</i>

Tabla 5.11: Ejemplos cualitativos del experimento *zero-shot* con glosas predichas (Pred.) y glosas de referencia (GT). Las referencias y predicciones se muestran en alemán tal como las produce el sistema.

Podemos destacar tres fallos comunes.

Primero un **fallo por escasez de glosas**: cuando el reconocedor produce secuencias muy cortas o incompletas (como *PRED BLEIBEN*), el modelo no dispone de suficiente información para construir una oración y responde correctamente con *Keine Übersetzung*. Este comportamiento es formalmente correcto, pero resulta inútil desde el punto de vista de la traducción.

Otro es la **ruptura de la instrucción**: en varios casos el modelo devuelve las glosas

casi sin traducir y añade comentarios explicativos en alemán sobre por qué no puede completar la tarea, incumpliendo directamente la instrucción de responder únicamente con el resultado. Este comportamiento refleja una tensión entre el *instruction following* del modelo y la dificultad de la tarea, cuando Qwen no puede producir una traducción coherente, su entrenamiento como asistente lo lleva a justificarse en lugar de intentarlo.

El tercer y más grave patrón es la **alucinación multilingüe**: en dos casos el modelo genera texto en chino mandarín, mezclando idiomas de forma abrupta a mitad de la respuesta. Esto apunta a una inestabilidad del modelo cuando recibe entradas que no encajan con sus distribuciones de entrenamiento. Las glosas en mayúsculas con morfología alemana pero sintaxis de lengua de signos constituyen un tipo de input inusual que puede activar patrones de otros idiomas del corpus de preentrenamiento. Este problema aparece tanto con glosas predichas como con glosas de referencia, lo que confirma que el origen es la naturaleza de las glosas como input, no a los errores del reconocedor.

Comparando con el trabajo Spotter+GPT, que emplea GPT-3.5-turbo y alcanza un BLEURT de 0,49 con su detector visual y 0,79 con glosas de referencia, nuestro sistema obtiene resultados similares en el escenario con glosas predichas (0,50 vs 0,49) pero notablemente inferiores en el escenario con glosas de referencia (0,53 vs 0,79).

Esto muestra que cuando la entrada es ruidosa (glosas predichas), ambos sistemas se comportan de forma comparable, lo que sugiere que el cuello de botella en ese escenario es el reconocedor, no el modelo de lenguaje. Sin embargo, cuando la entrada es perfecta (glosas de referencia), GPT-3.5-turbo aprovecha mucho mejor la información disponible, apuntando a que Qwen 14B tiene una capacidad de razonamiento *zero-shot* inferior.

Cabe señalar que la comparación directa entre ambos trabajos tiene limitaciones ya que Spotter+GPT evalúa sobre conjuntos de datos distintos a PHOENIX-2014T, por lo que las diferencias pueden reflejar parcialmente la dificultad específica del corpus.

En cualquier caso, la brecha entre el *zero-shot* y la línea base de mBART (22,48 en BLEU-4) justifica explorar el **finetuning** como estrategia para alinear el modelo de lenguaje con la tarea específica de traducción de lengua de signos.

5.6. Ablation study: Qwen2.5-1.5B

Esta sección analiza el impacto de las distintas decisiones de diseño sobre el rendimiento del sistema de traducción, manteniendo fijo el módulo de reconocimiento (SLR-*JointDropout*) en todos los experimentos. Estudiaremos tres modificaciones distintas: la modalidad de entrada a Qwen (*features* visuales vs. glosas predichas), la cobertura de los adaptadores LoRA (solo atención vs. atención y FFN) y la estrategia de *finetuning* (LoRA vs. *finetuning* completo).

5.6.1. Modalidad de entrada: *features* visuales vs. glosas predichas

El primer eje de ablación compara dos fuentes de información distintas para condicionar la traducción: las *features* visuales proyectadas por el VLMapper y las glosas

predichas por el módulo CTC como texto plano. Ambas modalidades parten del mismo reconocedor y, por tanto, contienen información equivalente sobre la secuencia de signos, pero la representan de formas radicalmente distintas. Las *features* visuales son vectores continuos de alta dimensión que preservan la incertidumbre del reconocedor, mientras que las glosas son una decisión discreta y que descarta toda la información de extra que pueda existir.

Entrada	Config. LoRA	Dev					Test				
		B1	B2	B3	B4	R	B1	B2	B3	B4	R
Visual	LoRA-Att	34.16	25.38	19.98	16.35	37.05	34.29	25.54	19.89	16.31	36.50
Visual	LoRA-All	28.50	18.95	13.99	11.17	27.43	27.42	18.40	13.45	10.73	25.53
Glosas	LoRA-Att	34.00	23.82	18.26	14.84	33.70	34.71	25.06	19.19	15.63	34.59
Glosas	LoRA-All	35.39	25.91	20.17	16.42	36.12	35.35	26.08	20.31	16.65	35.69

Tabla 5.12: Ablación sobre la modalidad de entrada y la cobertura de LoRA con Qwen2.5-1.5B. B1–B4: BLEU-1 a BLEU-4. R: ROUGE-L.

Los resultados muestran que ninguna de las configuraciones de esta ablación supera a mBART (22,48 en BLEU-4 en test), pero revelan patrones de comportamiento relevantes.

En primer lugar, las features visuales con LoRA-Att (16,31 BLEU-4) y las glosas con LoRA-All (16,65 BLEU-4) obtienen resultados prácticamente equivalentes, lo que sugiere que ambas modalidades contienen una cantidad similar de información útil para la traducción cuando se usa la configuración de LoRA adecuada.

En segundo lugar, la combinación de features visuales con LoRA-All produce el peor resultado de la tabla (10,73 BLEU-4), muy por debajo de las demás configuraciones. Este colapso apunta a que ampliar los adaptadores al bloque FFN cuando la entrada son features visuales continuas desestabiliza el aprendizaje: el modelo tiene dificultades para alinear simultáneamente el espacio visual y ajustar las capas de proyección del FFN con tan pocos datos de entrenamiento.

5.6.2. Impacto del *prompt*

Para aislar la contribución del prompt de instrucción (“*Übersetze diese Glossen ins Deutsche*” que significa “Traduce estas glosas al alemán”), se compara la configuración LoRA-All con features visuales con y sin prompt.

Prompt	Dev					Test				
	B1	B2	B3	B4	R	B1	B2	B3	B4	R
Con prompt	28.50	18.95	13.99	11.17	27.43	27.42	18.40	13.45	10.73	25.53
Sin prompt	40.62	30.33	23.92	19.58	39.83	42.21	31.69	24.98	20.55	40.01

Tabla 5.13: Impacto del prompt de instrucción en la configuración LoRA-All con features visuales. B1–B4: BLEU-1 a BLEU-4. R: ROUGE-L.

El resultado es sorprendente: eliminar el prompt mejora el BLEU-4 en casi 10 puntos

(10,73 $\rightarrow$ 20,55) y el ROUGE-L en más de 14 puntos (25,53 $\rightarrow$ 40,01). Una explicación plausible es que, durante el *finetuning* con LoRA-All, el modelo aprende a condicionar la generación directamente sobre los tokens visuales del prefijo. En ese contexto, el prompt textual no aporta información adicional y actúa como ruido que desplaza la atención del modelo de las features visuales hacia una señal textual redundante. Cuando el prompt desaparece, los tokens visuales ocupan toda la ventana de contexto relevante y el modelo puede explotarlos con mayor eficacia. Este resultado sugiere que, en el régimen de *finetuning* con suficiente capacidad de adaptación, el prompt de instrucción es prescindible e incluso contraproducente.

5.6.3. *Finetuning* completo vs. LoRA

Finalmente se compara el *finetuning* completo de Qwen2.5-1.5B con la aproximación LoRA, en las dos modalidades de entrada disponibles.

Entrada	Estrategia	Dev					Test				
		B1	B2	B3	B4	R	B1	B2	B3	B4	R
Visual	LoRA-Att	34.16	25.38	19.98	16.35	37.05	34.29	25.54	19.89	16.31	36.50
Visual	<i>Full</i>	38.98	28.96	23.03	18.99	38.98	39.26	29.07	22.74	18.68	37.04
Glosas	LoRA-Att	34.00	23.82	18.26	14.84	33.70	34.71	25.06	19.19	15.63	34.59
Glosas	<i>Full</i>	34.64	25.12	19.55	15.98	34.31	35.05	25.43	19.57	16.00	34.89

Tabla 5.14: Comparativa entre *finetuning* completo y LoRA-Att con Qwen2.5-1.5B. B1–B4: BLEU-1 a BLEU-4. R: ROUGE-L.

El *finetuning* completo supera a LoRA-Att en la modalidad visual (18,68 vs 16,31 en BLEU-4), pero la diferencia es más modesta de lo esperado: apenas 2,37 puntos pese a actualizar todos los parámetros del modelo. En la modalidad de glosas, la diferencia es prácticamente nula (16,00 vs 15,63), lo que sugiere que para entradas textuales discretas LoRA tiene capacidad suficiente para adaptar el modelo.

El coste del *finetuning* completo en términos de memoria y tiempo de entrenamiento (una 2-3h más) es significativamente mayor que el de LoRA, por lo que la ganancia obtenida no justifica su uso salvo en el caso de features visuales, donde la mayor flexibilidad paramétrica parece ayudar a alinear el espacio visual con el espacio lingüístico de Qwen.

5.6.4. Mejor configuración de Qwen2.5-1.5B

De todos los experimentos anteriores, la configuración que obtiene el mejor resultado es **LoRA-All sin prompt con features visuales** (20,55 BLEU-4, 40,01 ROUGE-L en test), que será la referencia para la comparativa con modelos de mayor tamaño en la sección siguiente. Aunque este resultado sigue siendo inferior a mBART (22,48 BLEU-4), la diferencia se ha reducido a menos de 2 puntos, lo que representa un avance significativo respecto al escenario *zero-shot*.

5.7. Escalado a Qwen2.5-7B

Una vez identificada la mejor configuración para Qwen2.5-1.5B, se evalúa si escalar el modelo de lenguaje a **Qwen2.5-7B** produce una mejora proporcional en la calidad de la traducción. Dado que el *finetuning* completo es inviable bajo las restricciones de memoria de una única A100 de 40 GB para un modelo de este tamaño, se emplea exclusivamente LoRA-Att con features visuales, la configuración más comparable a la mejor variante de 1.5B.

Modelo	Dev					Test				
	B1	B2	B3	B4	R	B1	B2	B3	B4	R
Qwen2.5-1.5B LoRA-Att	34.16	25.38	19.98	16.35	37.05	34.29	25.54	19.89	16.31	36.50
Qwen2.5-7B LoRA-Att	42.23	31.57	24.81	20.31	40.95	42.61	31.90	24.82	20.03	41.26

Tabla 5.15: Comparativa entre Qwen2.5-1.5B y Qwen2.5-7B con LoRA-Att y features visuales. B1–B4: BLEU-1 a BLEU-4. R: ROUGE-L.

El salto de 1.5B a 7B parámetros produce una mejora sustancial: el BLEU-4 pasa de 16,31 a 20,03 en test (+3,72 puntos) y el ROUGE-L de 36,50 a 41,26 (+4,76 puntos), con la misma configuración de LoRA y sin ningún cambio en el resto del pipeline. Esta mejora es consistente entre validación y test, lo que descarta que sea un artefacto de sobreajuste al conjunto de validación.

Resulta especialmente significativo que Qwen2.5-7B con LoRA-Att alcance un rendimiento comparable al *finetuning* completo de Qwen2.5-1.5B (20,03 vs 18,68 en BLEU-4), superándolo en más de un punto pese a actualizar un número de parámetros muy inferior (unos 40M vs. 1600M). Esto sugiere que la capacidad del modelo base es un factor más determinante que la estrategia de *finetuning* cuando el corpus de entrenamiento es reducido: un modelo más grande con menos parámetros adaptados generaliza mejor que un modelo más pequeño con todos sus parámetros actualizados.

No obstante, Qwen2.5-7B sigue sin alcanzar la línea base de mBART (22,48 BLEU-4), quedando a 2,45 puntos de distancia. La pregunta natural es si esta brecha se cerraría con un modelo aún mayor; sin embargo, realizar cualquier tipo de adaptación con Qwen2.5-14B excede la capacidad de memoria disponible en el entorno experimental, por lo que su evaluación queda restringida al escenario *zero-shot* presentado anteriormente.

5.8. Comparativa global y discusión

5.8.1. Resumen de resultados

La Tabla 5.16 recoge todos los sistemas evaluados, ordenados por BLEU-4 en test, para facilitar la comparación directa.

Sistema	Modelo	Entrada	Estrategia	Dev					Test				
				B1	B2	B3	B4	R	B1	B2	B3	B4	R
MSKA-mBART	mBART	Visual	<i>Full</i>	45.49	34.03	26.58	21.43	45.20	45.03	34.51	27.43	22.48	45.00
MSKA-Qwen-7B	Qwen2.5-7B	Visual	LoRA-Att	42.23	31.57	24.81	20.31	40.95	42.61	31.90	24.82	20.03	41.26
MSKA-Qwen-1.5B (sin prompt)	Qwen2.5-1.5B	Visual	LoRA-All	40.62	30.33	23.92	19.58	39.83	42.21	31.69	24.98	20.55	40.01
MSKA-Qwen-1.5B (<i>full</i>)	Qwen2.5-1.5B	Visual	<i>Full</i>	38.98	28.96	23.03	18.99	38.98	39.26	29.07	22.74	18.68	37.04
MSKA-Qwen-1.5B (LoRA-All + glosas)	Qwen2.5-1.5B	Glosas	LoRA-All	35.39	25.91	20.17	16.42	36.12	35.35	26.08	20.31	16.65	35.69
MSKA-Qwen-1.5B (LoRA-Att)	Qwen2.5-1.5B	Visual	LoRA-Att	34.16	25.38	19.98	16.35	37.05	34.29	25.54	19.89	16.31	36.50
MSKA-Qwen-1.5B (LoRA-Att + glosas)	Qwen2.5-1.5B	Glosas	LoRA-Att	34.00	23.82	18.26	14.84	33.70	34.71	25.06	19.19	15.63	34.59
MSKA-Qwen-1.5B (<i>full</i> + glosas)	Qwen2.5-1.5B	Glosas	<i>Full</i>	34.64	25.12	19.55	15.98	34.31	35.05	25.43	19.57	16.00	34.89
Spotter+Qwen (<i>zero-shot</i>)	Qwen2.5-14B	Glosas	—	—	—	—	—	—	12.11	4.12	1.38	0.46	9.85

Tabla 5.16: Comparativa global de todos los sistemas evaluados sobre PHOENIX-2014T. Mejor resultado en negrita. B1–B4: BLEU-1 a BLEU-4. R: ROUGE-L.

5.8.2. Discusión

Tres conclusiones principales emergen de la comparativa global.

mBART sigue siendo la referencia más sólida. Con 22,48 BLEU-4 y 45,00 ROUGE-L en test, ninguna configuración basada en Qwen logra superarlo. La ventaja de mBART no es sorprendente: su arquitectura *encoder-decoder* con atención cruzada está específicamente diseñada para tareas de traducción, y su preentrenamiento multilingüe incluye alemán de forma explícita y masiva. Qwen, en cambio, es un modelo causal de propósito general que debe aprender a comportarse como un traductor a partir de un corpus de apenas 7,096 muestras.

El tamaño del modelo importa más que la estrategia de *finetuning*. Qwen2.5-7B con LoRA-Att supera al *finetuning* completo de Qwen2.5-1.5B (20,03 vs 18,68 en BLEU-4), lo que indica que la capacidad representacional del modelo base es el factor limitante principal en este régimen de datos escasos. Bajo esta lógica, un modelo más grande como Qwen2.5-14B ajustado con LoRA podría cerrar o incluso superar la brecha con mBART, aunque su evaluación queda fuera del alcance de este trabajo por restricciones de computación.

El prompt de instrucción es contraproducente en el régimen de *finetuning*. El experimento sin prompt obtiene el segundo mejor resultado global (20,55 BLEU-4) pese a usar un modelo de solo 1.5B parámetros, superando al *finetuning* completo del mismo modelo. Esto sugiere que, cuando el modelo dispone de suficiente capacidad de adaptación mediante LoRA-All, el prompt textual compite con los tokens visuales por la atención del modelo en lugar de complementarlos.

5.8.3. Coste computacional

La Tabla 5.17 resume el coste computacional de cada sistema, medido en número de parámetros entrenables y consumo de memoria durante el entrenamiento. El uso de memoria se reporta mediante dos fuentes complementarias: la estimación proporcionada por **Weights & Biases** (*Wandb*), que monitoriza la memoria GPU reservada por el proceso, y la medición directa de **PyTorch** mediante `torch.cuda.max_memory_allocated`, que refleja la memoria efectivamente ocupada por tensores en el pico de uso. La diferencia

entre ambas cifras corresponde a la memoria reservada pero no utilizada por el gestor de memoria de CUDA.

Sistema	Estrategia	Parámetros (M)	VRAM Wandb (GB)	VRAM PyTorch (GB)
MSKA-mBART	<i>Full</i>	657.70	37.2	32.23
MSKA-Qwen-1.5B	LoRA-Att	17.43	36.0	13.12
MSKA-Qwen-1.5B	LoRA-All	73.86	38.4	14.11
MSKA-Qwen-1.5B	<i>Full</i>	1543.70	36.8	19.55
MSKA-Qwen-7B	LoRA-Att	40.37	32.8	28.88

Tabla 5.17: Coste computacional de los distintos sistemas. Parámetros en millones. VRAM Wandb: memoria GPU reservada por el proceso; VRAM PyTorch: pico de memoria efectivamente ocupada por tensores (`torch.cuda.max_memory_allocated`).

Varios aspectos resultan destacables.

En primer lugar, **mBART** es el segundo sistema con mayor número de parámetros entrenables (657,70M), consecuencia directa de su *finetuning* completo, y también el que más memoria efectiva consume (32,23 GB según PyTorch), lo que deja un margen muy reducido dentro del presupuesto de 40 GB de la A100.

En segundo lugar, las variantes **LoRA** de Qwen-1.5B reducen drásticamente los parámetros entrenables: LoRA-Att entrena solo 17,43M de parámetros ($\sim 1.1\%$ del total del modelo), mientras que LoRA-All amplía esta cifra a 73,86M al incorporar también las capas *feed-forward*. Sin embargo, la diferencia en consumo de memoria efectiva entre ambas variantes es modesta (13,12 vs. 14,11 GB), lo que indica que el coste en memoria no proviene de los adaptadores en sí, sino de los activaciones y gradientes del modelo base congelado.

En tercer lugar, el ***finetuning* completo de Qwen-1.5B** actualiza 1543,70M de parámetros (todos los del modelo más el VLMapper), pero su consumo de memoria (19,55 GB) es moderado gracias al tamaño reducido del modelo base, lo que hace viable su entrenamiento en una única GPU.

Finalmente, **Qwen-7B con LoRA-Att** actualiza apenas 40,37M de parámetros, pero su consumo de memoria efectiva (28,88 GB) es el segundo más alto de la tabla debido al mayor tamaño del modelo base que debe mantenerse en memoria durante el paso hacia adelante. Esta configuración ocupa el punto óptimo en la relación rendimiento-coste: con solo el $2,5\times$ de memoria respecto a Qwen-1.5B LoRA-Att, obtiene una mejora de +3,72 puntos de BLEU-4, la mayor ganancia individual de toda la batería de experimentos.

Capítulo 6

Conclusión y trabajo futuro

Un dato revelador es que el WER sobre el conjunto de entrenamiento se sitúa en torno al 7 %, frente al 27 % obtenido en dev y test.

Esta brecha de aproximadamente 20 puntos porcentuales es una señal clara de sobreajuste: el modelo memoriza con alta precisión las secuencias de glosas del corpus de entrenamiento pero generaliza de forma limitada a muestras no vistas.

Este fenómeno es esperable dado el tamaño reducido del corpus (7,096 muestras de entrenamiento) y la complejidad de la arquitectura.

El hecho de que ninguna de las técnicas de aumentación logre cerrar significativamente esta brecha refuerza la hipótesis de que el problema no es trivialmente soluble mediante perturbaciones sobre los keypoints, y apunta a la necesidad de datos adicionales o estrategias de regularización más agresivas como líneas de trabajo futuro.

Bibliografía

- [1] NECATI CIHAN CAMGOZ, OSCAR KOLLER, SIMON HADFIELD Y RICHARD BOWDEN, *Sign Language Transformers: Joint End-to-end Sign Language Recognition and Translation*, arXiv preprint, art. 2003.13830, 2020. Disponible en: <https://arxiv.org/abs/2003.13830>
- [2] HAO ZHOU, WENGANG ZHOU, YUN ZHOU Y HOUQIANG LI, *Spatial-Temporal Multi-Cue Network for Continuous Sign Language Recognition*, CoRR, vol. abs/2002.03187, 2020. Disponible en: <https://arxiv.org/abs/2002.03187>
- [3] BENJIA ZHOU, ZHIGANG CHEN, ALBERT CLAPÉS, JUN WAN, YANYAN LIANG, SERGIO ESCALERA, ZHEN LEI Y DU ZHANG, *Gloss-free Sign Language Translation: Improving from Visual-Language Pretraining*, arXiv preprint, art. 2307.14768, 2023. Disponible en: <https://arxiv.org/abs/2307.14768>
- [4] YUTONG CHEN, RONGLAI ZUO, FANGYUN WEI, YU WU, SHUJIE LIU Y BRIAN MAK, *Two-Stream Network for Sign Language Recognition and Translation*, arXiv preprint, art. 2211.01367, 2023. Disponible en: <https://arxiv.org/abs/2211.01367>
- [5] HEZHEN HU, WEICHAO ZHAO, WENGANG ZHOU Y HOUQIANG LI, *SignBERT+: Hand-Model-Aware Self-Supervised Pre-Training for Sign Language Understanding*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 45, no. 9, pp. 11221-11239, 2023. Disponible en: <http://dx.doi.org/10.1109/TPAMI.2023.3269220>
- [6] NADA E. ELSHAMI, AHMAD SALAH, AMR ABDELLATIF Y HEBA MOHSEN, *Toward Robust Human Pose Estimation Under Real-World Image Degradations and Restoration Scenarios*, Information, vol. 16, no. 11, art. 970, 2025. Disponible en: <https://www.mdpi.com/2078-2489/16/11/970>
- [7] HAO-SHU FANG, JIEFENG LI, HONGYANG TANG, CHAO XU, HAOWU ZHU, YULIANG XIU, YONG-LU LI Y CEWU LU, *AlphaPose: Whole-Body Regional Multi-Person Pose Estimation and Tracking in Real-Time*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022. Disponible en: <https://github.com/Fang-Haoshu/Halpe-FullBody>

- [8] CAMILLO LUGARESI, JIUQI TANG, HADON NASH, CHRIS MCCLANAHAN, EESHAN UBOWEJA, MICHAEL HAYS ET AL., *MediaPipe Holistic: Simultaneous Face, Hand and Pose Prediction, on device*, Google AI Blog, 2020.
- [9] Z. CAO, G. HIDALGO MARTINEZ, T. SIMON, S. WEI Y Y. A. SHEIKH, *Open-Pose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019.
- [10] L. ZHILIANG Y L. ZHUO, *Deep learning-based approaches for human pose estimation in interdisciplinary physics applications*, Scientific Reports, vol. 15, art. 42883, 2025. Disponible en: <https://doi.org/10.1038/s41598-025-26972-4>
- [11] ASHISH VASWANI, NOAM SHAZEER, NIKI PARMAR, JAKOB USZKOREIT, LLION JONES, AIDAN N. GOMEZ, LUKASZ KAISER E ILLIA POLOSUKHIN, *Attention Is All You Need*, arXiv preprint, art. 1706.03762, 2023. Disponible en: <https://arxiv.org/abs/1706.03762>
- [12] HAO-SHU FANG, SHUQIN XIE, YU-WING TAI Y CEWU LU, *RMPE: Regional Multi-person Pose Estimation*, Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017.
- [13] JIEFENG LI, CAN WANG, HAO ZHU, YIHUAN MAO, HAO-SHU FANG Y CEWU LU, *Crowdpose: Efficient crowded scenes pose estimation and a new benchmark*, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10863-10872, 2019.
- [14] NECATI CIHAN CAMGOZ, SIMON HADFIELD, OSCAR KOLLER, HERMANN NEY Y RICHARD BOWDEN, *Neural Sign Language Translation*, 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 7784-7793, 2018. Disponible en: <https://doi.org/10.1109/CVPR.2018.00812>
- [15] HAO ZHOU, WENGANG ZHOU, WEIZHEN QI, JUNFU PU Y HOUQIANG LI, *Improving Sign Language Translation with Monolingual Data by Sign Back-Translation*, IEEE/CVF International Conference on Computer Vision and Pattern Recognition (CVPR), 2021.
- [16] AMANDA DUARTE, SHRUTI PALASKAR, LUCAS VENTURA, DEEPTI GHADIYARAM, KENNETH DEHAAN, FLORIAN METZE, JORDI TORRES Y XAVIER GIRO-I-NIETO, *How2Sign: A Large-scale Multimodal Dataset for Continuous American Sign Language*, arXiv preprint, art. 2008.08143, 2021. Disponible en: <https://arxiv.org/abs/2008.08143>
- [17] F. MORILLAS-ESPEJO Y E. MARTÍNEZ-MARTÍN, *Sign4all: a Spanish Sign Language dataset*, Scientific Data, vol. 13, no. 1, 2026. Disponible en: <https://doi.org/10.1038/s41597-026-06872-6>
- [18] JIANYUAN GUO, PEIKE LI Y TREVOR COHN, *Bridging Sign and Spoken Languages: Pseudo Gloss Generation for Sign Language Translation*, arXiv preprint, art. 2505.15438, 2025. Disponible en: <https://arxiv.org/abs/2505.15438>

- [19] MO GUAN, YAN WANG, GUANGKUN MA, JIARUI LIU Y MINGZU SUN, *Multi-Stream Keypoint Attention Network for Sign Language Recognition and Translation*, arXiv preprint, art. 2405.05672, 2024. Disponible en: <https://arxiv.org/abs/2405.05672>
- [20] OZGE MERCANOGLU SINCAN Y RICHARD BOWDEN, *Spotter+GPT: Turning Sign Spottings into Sentences with LLMs*, Adjunct Proceedings of the 25th ACM International Conference on Intelligent Virtual Agents, pp. 1-6, 2025. Disponible en: <http://dx.doi.org/10.1145/3742886.3756708>
- [21] MATT POST, *A Call for Clarity in Reporting BLEU Scores*, arXiv preprint, art. 1804.08771, 2018. Disponible en: <https://arxiv.org/abs/1804.08771>
- [22] LEI SHI, YIFAN ZHANG, JIAN CHENG Y HANQING LU, *Decoupled Spatial-Temporal Attention Network for Skeleton-Based Action Recognition*, arXiv preprint, art. 2007.03263, 2020. Disponible en: <https://arxiv.org/abs/2007.03263>
- [23] KISHORE PAPINENI, SALIM ROUKOS, TODD WARD Y WEI-JING ZHU, *Bleu: a Method for Automatic Evaluation of Machine Translation*, Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311-318, 2002. Disponible en: <https://aclanthology.org/P02-1040/>
- [24] CHIN-YEW LIN, *ROUGE: A Package for Automatic Evaluation of Summaries*, Text Summarization Branches Out, pp. 74-81, 2004. Disponible en: <https://aclanthology.org/W04-1013/>
- [25] THIBAUT SELLAM, DIPANJAN DAS Y ANKUR P. PARIKH, *BLEURT: Learning Robust Metrics for Text Generation*, arXiv preprint, art. 2004.04696, 2020. Disponible en: <https://arxiv.org/abs/2004.04696>
- [26] ALEX GRAVES, SANTIAGO FERNÁNDEZ, FAUSTINO GOMEZ Y JÜRGEN SCHMIDHUBER, *Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks*, ICML 2006 - Proceedings of the 23rd International Conference on Machine Learning, vol. 2006, pp. 369-376, 2006. Disponible en: <https://doi.org/10.1145/1143844.1143891>
- [27] EIRINI MATHE, IOANNIS VERNIKOS, EVAGGELOS SPYROU Y PHIVOS MYLONAS, *Leveraging Artificial Occluded Samples for Data Augmentation in Human Activity Recognition*, Sensors, vol. 25, no. 4, art. 1163, 2025. Disponible en: <https://www.mdpi.com/1424-8220/25/4/1163>
- [28] CHU XIN, SEOKHWAN KIM, YONGJOO CHO Y PARK KYOUNG SHIN, *Enhancing Human Action Recognition with 3D Skeleton Data: A Comprehensive Study of Deep Learning and Data Augmentation*, Electronics, vol. 13, no. 4, art. 747, 2024. Disponible en: <https://www.mdpi.com/2079-9292/13/4/747>
- [29] DIAKO RUDD, *Understanding Kullback-Leibler (KL) Divergence*, Medium, 2024. Disponible en: <https://medium.com/@diako.rudd/understanding-kullback-leibler-kl-divergence-d77c976527c8>

- [30] YINHAN LIU, JIATAO GU, NAMAN GOYAL, XIAN LI, SERGEY EDUNOV, MARJAN GHAZVININEJAD, MIKE LEWIS Y LUKE ZETTLEMOYER, *Multilingual Denoising Pre-training for Neural Machine Translation*, CoRR, vol. abs/2001.08210, 2020. Disponible en: <https://arxiv.org/abs/2001.08210>
- [31] ABHINAV P. M., SUJAYKUMAR REDDY M Y OSWALD CHRISTOPHER, *Machine Translation with Large Language Models: Decoder Only vs. Encoder-Decoder*, arXiv preprint, art. 2409.13747, 2024. Disponible en: <https://arxiv.org/abs/2409.13747>
- [32] QWEN, AN YANG, BAOSONG YANG, BEICHEN ZHANG, BINYUAN HUI, BO ZHENG, BOWEN YU, CHENGYUAN LI, DAYIHENG LIU, FEI HUANG, HAORAN WEI, HUAN LIN, JIAN YANG, JIANHONG TU, JIANWEI ZHANG, JIANXIN YANG, JIAXI YANG, JINGREN ZHOU, JUNYANG LIN, KAI DANG, KEMING LU, KEQIN BAO, KEXIN YANG, LE YU, MEI LI, MINGFENG XUE, PEI ZHANG, QIN ZHU, RUI MEN, RUNJI LIN, TIANHAO LI, TIANYI TANG, TINGYU XIA, XINGZHANG REN, XUANCHENG REN, YANG FAN, YANG SU, YICHANG ZHANG, YU WAN, YUQIONG LIU, ZEYU CUI, ZHENRU ZHANG Y ZIHAN QIU, *Qwen2.5 Technical Report*, arXiv preprint, art. 2412.15115, 2025. Disponible en: <https://arxiv.org/abs/2412.15115>
- [33] QWEN TEAM, *Qwen2.5: A Party of Foundation Models*, Qwen Blog, 2024. Disponible en: <https://qwenlm.github.io/blog/qwen2.5/>
- [34] EDWARD J. HU, YELONG SHEN, PHILLIP WALLIS, ZEYUAN ALLEN-ZHU, YUANZHI LI, SHEAN WANG Y WEIZHU CHEN, *LoRA: Low-Rank Adaptation of Large Language Models*, CoRR, vol. abs/2106.09685, 2021. Disponible en: <https://arxiv.org/abs/2106.09685>
- [35] HAOTIAN LIU, CHUNYUAN LI, QINGYANG WU Y YONG JAE LEE, *Visual Instruction Tuning*, arXiv preprint, art. 2304.08485, 2023. Disponible en: <https://arxiv.org/abs/2304.08485>